実装し、観測し、速くし、正しさを確かめる



ライフゲームで学ぶ

はじめての Bend2

基礎文法からライフゲームの完成、並列計算と証明へ

本文・演習・Bend ソース・独立テスト付き

第 1 版　/　Bend 2.0.36 基準

本書の読み方と検証状態

本書で作るものは、**8×8 のトーラス型ライフゲーム（B3/S23）を 5 世代表示するターミナル・シミュレータ**です。完成版はsrc/08_life_terminal.bendに収録しています。ただし、初めから完成版を丸写しするのではありません。まず Bend 2 の型・関数・配列などを学びます。その後ライフゲームの規則を確認し、1 セルの更新、盤面と近傍の計算、世代更新、文字表示を順に組み合わせます。

要点

まず **Part I で Bend 2 の基礎**をゲームの設定から独立して学び、**Part II でライフゲームのルールと設計**を確認してから本体を組み立てます。Part III では入出力（I/O）や画面表示などのアプリケーション基礎を学び、Part IV では計算量、並列化、証明へ進みます。各章では「出力を予想する → 実行する → 改造する → 結果を記録する」を繰り返しましょう。最後に自分で新しいパターンを追加し、テストで確かめます。

注意

検証状態を区別してください。本書の Bend コードは、GitHub Actions 上の **Bend 2.0.37** で、意図的に誤った例を除く 26 ファイルの型検査に成功しました。逐次ライフゲームの 5 世代全セルの出力は独立した Python 参照実装と完全一致し、局所生死規則 18 ケースと Python の自動テスト 6 件にも合格しています。これは**型検査と特定の入力に対する実行結果の検査**であり、任意の盤面サイズに対する正しさを数学的に証明したことではありません。GUI 実行、GPU 上のライフゲーム本体、証明例の`--verdict` 検査は未確認・未完成のものを区別して説明します。

本書では、各 Part の見出しと章別目次を同じページにまとめました。また、章の冒頭に学習内容の見取り図を置き、必要な用語を登場する場所で説明します。基本機能の学習範囲は、最後の「Bend 2 基本機能監査表」で確認できます。

必修実装は、CPU 上で 5 世代を表示するターミナル・シミュレータです。**上級拡張**は、CPU でのマルチコア化、GPU 移植、ライフゲームに関する形式的な証明を扱う研究演習です。付属する独立の並列サンプルや証明例と、ライフゲーム本体への統合が完了した状態とは区別してください。

同梱ファイル

src/01…11.bend	Life 本体、並列・GPU 独立例、局所規則の証明例。
src/13…30.bend	型、リスト、高階関数、クロージャ、モナド、テンプレート、IO、App、Kind、Map などの補充例。
src/12_…INVALID.bend	アフィン性の違反を観察するための**意図的な誤り**。通常のビルド対象から外す。
scripts/reference.py	Python 製の独立オラクル。期待盤面を生成します。
scripts/check_bend.py	Bend 出力をオラクルと比較し、不一致や実行失敗で非ゼロ終了する統合テスト。
tests/test_reference.py	Python オラクル自体の 6 件の検査。
Bend2_Life_Textbook.tex	XeLaTeX 原稿。編集・再組版が可能です。

学習前に知っておきたい基本用語

プログラミングを始めたばかりの方も、本書を読み進められるようにします。ここでは、まず**プログラム**が「コンピュータに行わせる処理を書いたもの」であることを確認します。そのプログラムを人が読んだり編集したりできる文字列が**ソースコード（source code）**です。難しい言葉をすべて覚える必要はありません。コードに出てきたときに、このページへ戻ってください。

用語	意味と本書での使い方
値（value）	数値の `3` や文字列の`"Life"` など、計算の対象になるものです。
変数（variable）	値に付ける名前です。`x` や `board` などを使います。Bend 2 では利用回数に制約がある場合があります。
式（expression）	計算して値になる記述です。たとえば `(2 + 3 : U32)` です。
関数（function）	入力を受け取り結果を返す処理です。`calculate` は整数の計算を行います。
引数（argument）	関数に渡す具体的な値です。`calculate(2,3)` の `2` と `3` が引数です。関数を宣言するときの `x` などの名前は**仮引数（parameter）**と呼びます。
返り値（return value）	関数が計算の結果として返す値です。`calculate(2,3)` の返り値は `13` です。
型（type）	値の種類と、可能な操作を表す情報です。`U32` は 32 ビット符号なし整数、`String` は文字列の型です。
配列（array）	複数の値を順番に並べ、位置を指定してアクセスする構造です。64 要素に固定された仕組みではありません。
添字（index）	配列の要素がどの位置にあるかを示す番号です。最初の位置を 0 として数えます。
再帰（recursion）	関数が自分自身を呼ぶ処理です。終わる条件を設ける必要があります。
副作用（side effect）	値を返すだけでなく、画面表示やファイル書き込みなど外部へ働きかけることです。
コンパイル（compilation）	ソースコードを、実行できる形式などへ変換する工程です。**型検査（type checking）**は、各式の型が合っているかを調べる検査です。
テスト（test）	期待した入力と出力を用意し、実際の動作が一致するかを調べることです。すべての入力に対する数学的な証明とは異なります。

要点

本書では、日常的な説明には一般に使われている日本語を優先します。Bend の型名や構文（`Kind`、`Data`、`IO` など）はそのまま記しますが、登場する場所で意味を説明します。

Contents

本書の読み方と検証状態 **ii**

学習前に知っておきたい基本用語 **iv**

I Bend 2 の基本文法を学ぶ **1**

1 環境構築と最初の実行 **2**

1.1 導入前に確認すること 2

1.2 Bend のプログラムを実行する 2

1.3 型エラーの表示を確認する 3

2 値とデータ型を理解する **4**

2.1 値と型の対応を読む 4

2.2 選択肢を新しい型として定義する 4

3 関数を定義して数値を計算する **6**

3.1 引数と返り値の型 6

3.2 演算でつまずく場所 6

4 パターンマッチで場合分けをする **8**

4.1 データ型の形を見て処理を選ぶ 8

4.2 条件に応じて処理を分ける方法 8

4.3 真の場合と偽の場合を追う 9

5 配列の操作とアフィン性 **10**

5.1 配列から値を取り出す基本操作 10

5.2 値を使える回数を表す記号 11

6 List と Maybe：再帰的データと失敗を扱う **12**

6.1 配列とリストを区別する 12

6.2 Maybe と Result の違い 12

7 高階関数とクロージャを使う 14
7.1 関数も引数にできる . . . 14
7.2 クロージャは外側の値を覚えている . . . 14
7.3 一度使う関数と、何度も適用する処理を区別する . . . 15

8 テンプレートとコンパイル時の引数 16
8.1 ~ は実行時のクロージャではない . . . 16
8.2 コンパイル時に処理を選択する . . . 16

9 数値・文字列・演算子と標準ライブラリ 17
9.1 主要な値の型 . . . 17
9.2 演算子の型付けとビット演算 . . . 17
9.3 Base を読む . . . 17

10 モジュールと複数ファイルのプログラム 19
10.1 `import ./... as M` . . . 19
10.2 パッケージ配布と名前の扱い . . . 19

II ライフゲームを設計して組み立てる 21

11 ライフゲームのしくみと制作するもの 22
11.1 何を作るか . . . 22
11.2 生死規則 B3/S23 . . . 22
11.3 受け入れ条件 . . . 23

12 一次元配列で 8×8 盤面を表す 24
12.1 生死規則を関数で表す . . . 24
12.2 二次元の盤面を一次元配列に並べる . . . 25
12.3 すべてのセルを 0 で初期化する . . . 25
12.4 配列の添字と盤面の座標を区別する . . . 25

13 グライダーを初期配置する 26
13.1 小さなパターンを正確に配置する . . . 26
13.2 盤面を数える . . . 27

14 トーラスの座標と 8 近傍 28
14.1 盤面の端を越えた座標を計算する . . . 28
14.2 周囲 8 セルを読み取り生存数を数える . . . 28
14.3 境界にいるセルを手計算する . . . 30

15 全セルを同期的に更新する **31**
15.1 更新に必ず二枚の盤面を使う 31
15.2 再帰処理が終わる条件を決める 31
15.3 ソースを読む順番 33
15.4 別の実装と結果を比較する 33

16 複数世代の実行とターミナル描画 **35**
16.1 世代を繰り返す関数 35
16.2 出力を作る関数は更新と分ける 35
16.3 実行する 38
16.4 期待される世代 4 38

17 テスト設計と失敗の切り分け **39**
17.1 プログラムの正しさをテストで確認する 39
17.2 参照実装を動かす 39
17.3 用意された独立テスト 6 件 40

18 完成版の設計をレビューする **41**
18.1 関数が担う責務 41
18.2 保守性チェック 41

III アプリケーション基礎を広げる **42**

19 モナドと `do` 構文を理解する **43**
19.1 値を返す計算と、失敗し得る計算の違い 43
19.2 手続き型の失敗判定を `Maybe` に置き換える 43
19.3 モナドとは何を共通化した仕組みか 44
19.4 `bind` を直接書いてみる 44
19.5 モナドの規則と `Result` への発展 44
19.6 同じ `do` で入出力をつなぐ 45

20 画面出力とファイル操作を扱う **46**
20.1 値の計算と効果の実行 46
20.2 入力・引数・ファイル・失敗 46
20.3 ファイルを実際に扱う 47
20.4 Life での実習 47

21 複数の入出力処理を進める方法 **48**
21.1 `IO.fork` と `IO.join` 48

21.2 キャンセル・タイムアウト・共有状態 48

22 ライフゲームを画面に表示する準備 50
22.1 純粋な描画とイベント更新 50
22.2 Life の画面表示への発展 50

23 Bend のプログラムを別の環境で動かす 52
23.1 標準 IO の外に出るとき 52
23.2 複数のターゲットを知る 52
23.3 公開プロジェクトでの安全性 52

IV Bend の強みを掘り下げる 54

24 再帰の停止性と計算量 55
24.1 停止性は速度の話ではない 55
24.2 再帰を安全に終わらせる引数の指定 55
24.3 更新処理の計算量を見積もる 55

25 アフィン性・所有権・配列の更新 56
25.1 一度しか使わないとは何か 56
25.2 複数のセルを読むときの処理 56
25.3 ひとつの盤面を並列更新する際の課題 56

26 CPU 並列：純粋な並列呼び出しを理解する 58
26.1 Bend で並列計算を記述する 58
26.2 実行形態を分けて測る 58
26.3 ライフゲームへの設計移植 59

27 GPU 呼び出しと実行レーン 60
27.1 ! の意味 60
27.2 単なる GPU 化で速くなるとは限らない 61
27.3 ライフゲームの GPU 化のための要件定義 61

28 性能評価：速さを数字で扱う 62
28.1 誤解されやすい三つの時間 62
28.2 比較表を作る 62
28.3 スピードアップと効率 62
28.4 実験ノートの標準書式 62

29 型を使って条件を表す 64

29.1 値によって変わる型を理解する 64
29.2 セルの値を型で制限する設計 64

30 規則の正しさを証明する 65
30.1 証明したいことを型として書く 65
30.2 証明している命題を日本語で読む 66
30.3 意図的に破ってみる 66

31 局所証明から盤面全体の正しさへ 67
31.1 未証明の箇所を明示する 67
31.2 全体仕様への分解案 67
31.3 law 駆動の開発 67

32 上級プロジェクト：可変盤面と GPU 化を設計する 68
32.1 どの順番で拡張するか 68
32.2 可変サイズにするときの具体的な問題 68
32.3 ハロー付きタイル化の擬似コード 68
32.4 卒業制作の到達基準 69

33 量付き Kind と汎用的な型を整理する 70
33.1 Type と Data の一般形 70
33.2 配列の所有権と複製の条件 70

34 証明の基本操作：帰納法・書き換え・反証 72
34.1 等式の証明を一歩進める 72
34.2 存在と否定を表す・未完成の証明を扱う 72
34.3 LAWS/PROOF の二ファイル構成 73

V 解答・補足資料 74

35 重要演習の解答と検証方法 75
35.1 18 ケースの局所規則 75
35.2 代表的な添字 75
35.3 グライダーの変位 75
35.4 近傍読み出し回数 75
35.5 典型的なバグ 76

36 環境・コマンド・参照先 77
36.1 コマンド早見表 77
36.2 主要資料 77

36.3 検証ログの書式 77
36.4 最後の総合チェック 78

37 Bend 2 基本機能監査表 79
37.1 監査基準と凡例 79
37.2 網羅性の限界：基本構文と全 API は別 80
37.3 追試チェックリスト（Bend を導入したら） 80

PART I

Bend 2 の基本文法を学ぶ

この Part の目次

1 環境構築と最初の実行 2

2 値とデータ型を理解する 4

3 関数を定義して数値を計算する 6

4 パターンマッチで場合分けをする 8

5 配列の操作とアフィン性 10

6 List と Maybe：再帰的データと失敗を扱う 12

7 高階関数とクロージャを使う 14

8 テンプレートとコンパイル時の引数 16

9 数値・文字列・演算子と標準ライブラリ 17

10 モジュールと複数ファイルのプログラム 19

要点

この Part では、**Bend 2 の言語仕様**を学びます。最初にプログラムの実行方法とデータ型を確認し、関数、場合分け、配列と所有権へ進みます。その後、リスト、関数の再利用、モジュールを学びます。各章では小さな独立したプログラムを実行して、言語機能そのものを理解しましょう。**具体的な制作物の設計と実装は Part II から始めます。**

Chapter 1

環境構築と最初の実行

本章では、**ターミナル（terminal）**という文字入力の画面から Bend を起動します。コマンドを一つ実行し、表示が変わるところまで進みましょう。

1.1　導入前に確認すること

Bend 2 は旧 Bend 1 とは別物として扱います。2026 年 10 月 16 日時点で GitHub の公式リリース一覧における最新タグは `v2.0.36`。本書の例もこれを基準とします。Windows は WSL の Linux 環境で進めるとよいでしょう。macOS や Linux ではターミナルを使います。GPU がなくても基礎章と証明章は学習できます。

```
curl -fsSL https://bend-lang.com/install.sh -o install-bend.sh
# Inspect install-bend.sh before execution.
sh install-bend.sh
export PATH="$HOME/.bend/bin:$PATH"
bend version
bend guide
bend base
```

Listing 1.1: 導入、動作確認、一次資料の表示

`bend version` の値と表示されたパスをVERSION_LOG.mdへ記録します。インストール用シェルを内容確認せず実行しません。`bend update` がある環境では勝手に更新した後の資料と旧コードを混ぜず、バージョンを記録してから学習します。

1.2　Bend のプログラムを実行する

```
import Base

def main() -> IO(Unit):
  do IO<Unit>:
    IO.print("Hello, Bend 2")
```

Listing 1.2: はじめての Bend 2（01_hello.bend）

`import Base` は標準ライブラリの参照であり、`def main() -> IO(Unit):` は端末表示という副作用を伴う入口を宣言します。`do IO<Unit>:` で順番に効果を行い、`IO.print` を実行します。「純粋な値を返す関数」と「外界へ働きかける関数」は区別されます。

```
bend src/01_hello.bend
bend src/01_hello.bend --check-only
bend src/01_hello.bend -o build/hello
./build/hello
```

Listing 1.3: 確認からネイティブ生成まで

ネイティブ生成には適切な Clang が必要です。GPU を呼ぶコードでは要求バージョンが高くなり、Linux では CUDA 12、macOS では Metal の要件があります。**Bend インタプリタの所要時間とネイティブ実行時間を同列で比較しません**。

1.3 型エラーの表示を確認する

`IO.print` の文字列を数値に変えてみましょう。次に型注釈を消してみます。さらに返り値の型を `U32` にして表示命令を残し、どのエラーが出るか観察します。エラーメッセージのうち **expected** と **observed** がどの式を指しているかを説明します。

ハンズオン／確認問題

(1) `Hello, Bend 2` の文字列を変更して実行してみましょう。(2) `bend guide shaders` を実行し見出しを確認してみましょう。(3) 導入版・OS・CPU/GPU・Clang の情報をVERSION_LOG.mdに保存してみましょう。**合格条件**: ソースを編集して標準出力を変えられること。

Chapter 2

値とデータ型を理解する

プログラムを書く前に、**値（value）**と**型（type）**を区別しましょう。Bend 2 の型は「どんな値を表し、何をしてよいか」を決めます。

2.1　値と型の対応を読む

データ型（data type）とは、値の種類を定めるものです。例えば `42` は数値、`"hello"` は文字列です。

Bend 2 で頻出する型を先に見ておきます。

型	値の例	用途
`Nat`	`3n`	0 以上の自然数を表します。
`U32`	`42`	0 から $2^{32}-1$ までの符号な
`Bool`	`True{}`	真または偽を表します。
`String`	`"hello"`	文字列を表します。
`F32`	32 ビット浮動小数点	小数を近似して扱う数値型で

同じ数字に見えても `3n` と `3` は異なる型の値です。Bend 2 は型の暗黙変換が少ないため、型を意識してコードを読む必要があります。**型注釈（type annotation）**は、値や式の型を明示する記法です。例えば `(3 + 4 : U32)` は、`U32` 型の足し算を表します。

2.2　選択肢を新しい型として定義する

既存の型だけでは表しにくい値は、**ユーザー定義型（user-defined type）**を作って表現できます。次の例では、**コンストラクタ（constructor）**の `Idle{}` と `Busy{}` という二種類だけの状態を定義します。コンストラクタとは、**その型の値を組み立てるための名前**です。

```
import Base

# This custom datatype has exactly two constructors.
type Status is Data:
  Idle{}
  Busy{}

def main() -> Status:
  Idle{}
```

Listing 2.1: 二つの状態を持つデータ型（28_data_types.bend）

最初の `type Status is Data:` が型の宣言、続く二行が作れる値の候補です。最後の `main` はプログラ

ムの入口で、詳しい関数の書き方は次章で学びます。ここでは、`Idle{}` が `Status` 型の値になることを読み取りましょう。

要点

代数的データ型（algebraic data type; ADT）は、候補を列挙したり、候補ごとに別のデータを持たせたりできる型です。例えば `Some{value}` と `None{}` で「値がある／ない」を表す `Maybe` もその一例です。`is Data` は値を再利用できる分類を表しますが、所有権とあわせて後の章で扱います。

ハンズオン／確認問題

`Status` に `Paused{}` を追加してみましょう。`Idle{}` と `Busy{}` 以外の状態を、型を変えずに作れるか考えてみましょう。

Chapter 3

関数を定義して数値を計算する

関数（function）、**引数（argument）**、**返り値（return value）**を短い数値計算で確認します。特定のアプリケーションに依存しない例で、型注釈と式の読み方を身につけましょう。

3.1　引数と返り値の型

Bend では `def f(x: T) -> R:` の形で関数を宣言します。戻り値は最後の式であり、Python のように常に `return` が必要なのではありません。`U32` は 0 以上の整数を 32 ビットに格納する型です。`Nat` は 0、1、2、……という**自然数（natural number）**の型で、再帰や型の添字に使います。`3n` は `Nat` のリテラルで、`3` とは扱いを区別します。

```
import Base

# Basic arithmetic independent of any particular application.
def calculate(+x: U32, y: U32) -> U32:
  (x * 5 + y : U32)

def main() -> U32:
  calculate(2, 3)
```

Listing 3.1: 整数の乗算と加算（02_arithmetic.bend）

例の関数 `calculate(x,y)` は $5x + y$ を計算します。`calculate(2,3)` の結果は 13 です。`(x * 5 + y : U32)` では、計算式の後ろに型を示しています。`+x` の `+` は「値を複数回使える」と示す**使用量（quantity）**の指定であり、この短い例では必須ではありません。詳しくは配列とアフィン性の章で扱います。

3.2　演算でつまずく場所

整数加減算の演算子だけを式として書かず、`(x + y : U32)` のように型を与える。`Nat` と `U32` の暗黙変換を当てにしません。`U32.to_nat`・`U32.from_nat` 等は `bend base` で確認します。`U32` には範囲の限界があり、任意精度整数として扱ってはいけません。

要点

数字だけで動作確認する `main() -> U32` は、値の正規化と表示が行われる経路になります。大きな仕事量・並列性能の測定では`-o` で生成した実行ファイルを用いる。

ハンズオン／確認問題

`calculate(0,0)`、`calculate(1,2)`、`calculate(2,3)` の結果を予想してから実行してみましょう。式の 5 を 10 に変えると何が変わるでしょうか。また、定数 5 を引数 `factor: U32` に変える方法も考えてみましょう。

Chapter 4

パターンマッチで場合分けをする

Bend 2 では、条件に応じて処理を分けるときに**パターンマッチ（pattern matching）**を使います。ここでは、真偽値に応じて二つの数字から一つを選ぶ、一般的な例で基本形を学びます。

4.1 データ型の形を見て処理を選ぶ

前章で、`Status` には `Idle{}` と `Busy{}` という値の候補があることを学びました。**パターンマッチ（pattern matching）**は、**値がどの形に当てはまるか**を調べ、形ごとに処理を分ける仕組みです。`Bool` の `True{}` ／ `False{}` も同じ考え方で処理できます。

```
import Base

# A custom type restricts the set of valid values.
type Status is Data:
  Idle{}
  Busy{}

def to_u32(status: Status) -> U32:
  match status:
    case Idle{}:
      0
    case Busy{}:
      1

def main() -> U32:
  to_u32(Busy{})
```

Listing 4.1: ユーザー定義型をパターンマッチする例（13_status_adt.bend）

`case Idle{}`: なら 0、`case Busy{}`: なら 1 を返します。ここでは数値を返していますが、選択肢の種類が増えたら対応する場合分けも追加する必要があります。

4.2 条件に応じて処理を分ける方法

たとえば「条件が真なら 10、偽なら 20 を返す」という処理には、条件による場合分けが必要です。`match` を使うと、それぞれの条件に対応する処理を明確に書けます。

Bend 2 の公開ガイドには通常の `if ... else` 構文がなく、`match` と `True{}` ／ `False{}` を使います。特に**計算した式を直接 match できない**制限があります。たとえば比較式をそのまま `match` せず、

計算結果を引数として受け取った関数の中で場合分けします。

```
import Base

# General-purpose conditional choice; no Life-specific rules.
def choose(flag: Bool, first: U32, second: U32) -> U32:
  match flag:
    case True{}:
      first
    case False{}:
      second

def main() -> U32:
  choose(True{}, 10, 20)
```

Listing 4.2: 真偽値による場合分け（27_match_basics.bend）

choose に渡す真偽値 flag が True{} なら最初の数値を、False{} なら次の数値を返します。match は**引数として受け取った値**に対して行います。条件が何を意味するかは呼び出す側が決めるため、さまざまな処理で再利用できます。

4.3　真の場合と偽の場合を追う

条件	第 1 候補	第 2 候補	返り値
True{}	10	20	10
False{}	10	20	20
True{}	5	9	5

ハンズオン／確認問題

choose(True{},10,20) と choose(False{},10,20) の返り値を予想して確かめてみましょう。第 1 候補と第 2 候補を変えて結果を比較してみましょう。比較式 U32.is_eq の結果を choose へ渡す使い方も試してみましょう。

Chapter 5

配列の操作とアフィン性

ここでは**配列（array）**を学びます。配列とは、複数の値を順番にまとめ、位置を指定して扱うデータ構造です。あわせて、Bend 2 の**アフィン性（affinity）**という値の使用方法に関する制約を学びます。

5.1 配列から値を取り出す基本操作

配列の要素を読むには、`Array.get` を使います。ここで一つだけ、普通の配列操作と違う点があります。Bend 2 では、通常の配列は所有する値として扱われ、操作に渡すと元の変数をそのまま再利用できません。そこで `Array.get` は、**取り出した要素**だけでなく、**その後も使うための配列**も組で返します。まずこの形を見てから、なぜ所有権が必要なのかを確認しましょう。

`Array<U32>` は、32 ビット符号なし整数を格納する配列型です。**種類（kind）**とは「型を分類するための分類」で、`Type` は値を自由に複製できない種類、`Data` は複製を許せる種類を表します。`Array<U32>` は `Type` に属するため、同じ配列を二つの処理に勝手に渡せません。読み取るときは `Array.get` が**組（pair）**として「続けて使う配列」と「読み取った値」の両方を返します。こうすることで同じ配列を勝手に何度も使わず、読み取り後の配列を再び明示して処理できます。

```
import Base

def read_value(r: Array<U32> & U32) -> U32:
  (a, value) = r
  value

def main() -> U32:
  a = [0 : U32*8n]
  a = Array.set(U32, a, 5, 42)
  read_value(Array.get(U32, a, 5))
```

Listing 5.1: 書き換えと読み出し（04_array.bend）

`[0 : U32*8n]` は、0 を 8 個並べて作る配列です。**8 要素という長さはこのサンプルが選んだ値**であり、Bend 2 の配列が 8 要素に限定されるわけではありません。`Array.set(U32,a,5,42)` は、添字 5 に 42 を書き込みます。続く `Array.get` は、`(a,value)` という組を返します。ただし計算結果をその場で直接分解せず、`read_value` 関数の引数 `r` として受け取り、関数の中で分解しています。最後は `value` だけを結果とするので、`a` は破棄されます。Bend は**アフィン**（高々 1 回の使用）であり、破棄は許されます。後出の「配列操作の糖衣構文」では `a[i]` と `a[i] <- v` も紹介します。

注意

Array は +a を宣言して何度も使うことができません。src/12_affine_probe_INVALID.bendはこの失敗を観察する専用ファイルであり**正しくないことが正常**。普通の実行対象には含めない。

5.2 値を使える回数を表す記号

記法	解釈
x: T	変数 x を高々 1 度使う。使わなくてもよい。
+x: T	T が Data なら何度も使えます。参照カウント等のコストは無視できません。
-A: Data	消去される型引数。計算結果の型検査には現れるが実行時には不要。
Array<T>	Type に属し、通常はコピー可能な Data ではありません。

ハンズオン／確認問題

Array.get の (array,value) を (value,array) と逆にするとどうなるか。Array.set した配列を使わず、古い配列を再利用すると何が問題か。エラーの全文を記録し、型エラーと規則違反を区別してみましょう。

Chapter 6

List と Maybe：再帰的データと失敗を扱う

配列以外のデータの表し方を学びます。**リスト（list）**は要素を先頭からつなぐ構造で、Maybe は「結果があるかどうか」を型に含める仕組みです。

6.1 配列とリストを区別する

Array は長さが 2 の冪などの制約を持つ所有権付き領域であり、List<A> は Nil{} または Con{h,t} で構成される帰納的な鎖です。処理する数値の並びや、順番に発生するイベントはリストで表現できます。

```
import Base

# A List is an inductive structure. The tail is smaller.
def count(xs: List<U32>) -> Nat:
  match xs:
    case Nil{}:
      0n
    case Con{h, t}:
      (1n + count(t) : Nat)

def main() -> Nat:
  count([1, 2, 3, 4, 5])
```

Listing 6.1: リストを構造再帰で処理する（14_list_basics.bend）

6.2 Maybe と Result の違い

Maybe<A> は値が Some{value} か None{} かを表します。Result は成功または失敗の情報を保持する用途に適します。文字列を数値に変換する場面では、0 を「解析失敗」の印と兼用せず、失敗を型で表す方が安全になります。Maybe の正確な量パラメータは bend base Maybe で確認します。

要点

再帰関数は**停止性を証明できる構造的な減少**が重要です。List では Con の tail、Nat では 1n+p の p が自然な減少引数になります。

ハンズオン／確認問題

(1) 整数を数個並べた `List<U32>` の長さを数えてみましょう。(2) 空リストでは長さが 0 になることを確かめてみましょう。(3) 数値の解析に失敗した場合を `Maybe` で表現する方法を考えてみましょう。

Chapter 7

高階関数とクロージャを使う

手続き型言語では、関数を「処理を呼び出す手段」と考えることが多いでしょう。ここでは、**関数そのものを値として受け渡す**考え方を学びます。その際に必要になる**高階関数（higher-order function）**と**クロージャ（closure）**を区別します。

7.1　関数も引数にできる

高階関数とは、**関数を引数として受け取るか、結果として関数を返す関数**です。例えば `apply` は「どの処理をするか」を引数 `f` として受け取り、入力 `x` に適用します。`U32 -> U32` は、`U32` を受け取り `U32` を返す**関数型（function type）**です。

```
import Base

# A higher-order function takes a function as its input.
def apply(f: U32 -> U32, x: U32) -> U32:
  f(x)

def main() -> U32:
  apply(x => (x + 2 : U32), 40)
```

Listing 7.1: 関数を引数に渡す（29_higher_order.bend）

`x => (x + 2 : U32)` は**無名関数（anonymous function）**です。名前を付けずにその場で関数を作ります。この例では 40 を受け取り 42 を返します。手続き型言語の「処理の分岐を増やす」設計とは異なり、**共通の処理手順を残したまま、実際の計算だけを差し替える**ことができます。

7.2　クロージャは外側の値を覚えている

無名関数が外側の変数を使う場合、関数本体だけでは結果を決められません。そこで、**クロージャは関数と、その関数が利用する外側の値（捕捉した値）をまとめたもの**として動きます。

```
import Base

# A captured function is affine, even if the captured value is Data.
def adder(k: U32) -> U32 -> U32:
  x => (x + k : U32)

def main() -> U32:
  add2 = adder(2)
```

```
add5 = adder(5)
add5(add2(1))
```

Listing 7.2: 異なる値を捕捉する二つのクロージャ（15_closure.bend）

`adder(2)` の結果である `add2` は、`k=2` を覚えた関数です。`add2(1)` は 3 になり、`add5(3)` は 8 になるので、全体では 8 を返します。`adder` は**関数を返す高階関数**にもなっています。`U32.add(2)` のように一部の引数を先に与える**部分適用（partial application）**もクロージャの一例です。

注意

Bend 2 では、**クロージャはアフィン**です。つまり、同じクロージャを呼べるのは高々 1 回です。捕捉した `U32` が複製可能でも、クロージャ自体を何度も呼ぶことはできません。一方、`adder` のようなトップレベルの関数定義は繰り返し呼び出せます。これは Python や JavaScript の一般的なクロージャとは重要な違いです。

7.3　一度使う関数と、何度も適用する処理を区別する

`apply(f, x)` は `f` を一度使うので、通常の高階関数として書けます。しかし、同じ `f` を二度使う `f(f(x))` は、一般のアフィンな関数引数では書けません。次章では、**コンパイル時にコードを埋め込むテンプレート**を学び、この違いを確かめます。

要点

高階関数は「関数を受け取る／返す」という**役割**の名前です。**クロージャ**は「関数と捕捉した値」を一体として扱う**値**の名前です。同じものを指す言葉ではありません。

ハンズオン／確認問題

`29_higher_order.bend` で無名関数を `x => (x * 2 : U32)` に変えて結果を予想してみましょう。続いて `adder(3)` を作り、捕捉する値によって結果がどう変わるか確認してみましょう。同じクロージャを二度呼ぶ例は別ファイルにして、型検査が拒否する理由も考えてみましょう。

Chapter 8

テンプレートとコンパイル時の引数

「実行するたびに値を受け取る」のと「コンパイル時に処理を埋め込む」のは違います。ここでは**テンプレート（template）**を後者として理解します。

8.1 ~ は実行時のクロージャではない

テンプレート・パラメータ ~f はコンパイル時にコードとして代入されます。関数適用が複数必要なときでも、アフィンなクロージャを無理にコピーしません。**各テンプレート引数の組み合わせごとにインスタンス化**されるため、コードサイズとのトレードオフがあります。

```
import Base

# ~f is a compile-time function argument, unlike a closure.
def twice(~f: U32 -> U32, x: U32) -> U32:
  f(f(x))

def main() -> U32:
  twice(~(x => (x + 1 : U32)), 40)
```

Listing 8.1: 同じ処理を 2 度行うテンプレート（16_template.bend）

~(x => (x + 1 : U32)) は**呼び出し元の局所変数を捕捉しない閉じた構文**でなければなりません。テンプレートの宣言順序にも制約があります。List.map など Base の定義を bend base で読むと利用例が分かります。

8.2 コンパイル時に処理を選択する

処理 f を通常の引数にするか、~f としてテンプレート引数にするかは、実行時に選ぶ必要があるか、コンパイル時に決まってよいかで判断します。型と使い回しの制約も比較してみましょう。

ハンズオン／確認問題

twice を thrice へ拡張してみましょう。テンプレートと普通のクロージャの寿命・生成コードの違いを、実行時とコンパイル時に分けて説明してみましょう。

Chapter 9

数値・文字列・演算子と標準ライブラリ

数値型と文字列型は、用途ごとに使い分けます。まず整数と小数の違い、演算子が受け取る型、標準ライブラリの調べ方を整理します。

9.1 主要な値の型

Bend 2 の基本的な数値型は Nat、U32、F32 です。F32 は小数を近似的に表す**浮動小数点数（floating-point number）**で、ビット単位の演算や丸めに注意します。Bool、Cmp、Char、String も使用します。現行の公式ガイドは **F64 や I64 が最初から使える**とは述べていない。

```
import Base

# Parenthesized operators carry a type annotation.
def next_digit(+i: U32) -> U32:
  ((i + 1 : U32) % 10 : U32)

def main() -> String:
  "next digit = " ++ U32.show(next_digit(9))
```

Listing 9.1: 整数演算と文字列化（17_numbers_strings.bend）

9.2 演算子の型付けとビット演算

(a + b : U32) のように、型注釈は**その演算を囲う式自身**に置きます。&& ・ || は Bool、++ は String。.&.、.|.、.^. や << などは U32 のビット演算で、一般の数学的な等式と区別します。F32 のビット精度を形式的に証明できると期待しません。

9.3 Base を読む

bend base --types は型、bend base U32 は整数、bend base String は文字列、bend base Map は連想写像の API を調べる手段となります。Map は文字列キーの辞書、Set は集合です。学習では**新しい名前を勝手に推測せず、インストール版の Base で引数・返却型を確認する**。

```
import Base

# A Map is a string-keyed associative structure with one owner.
```

```
def take_value(result: Map<&2, U32> & U32) -> U32:
  (dictionary, number) = result
  number

def main() -> U32:
  m = Map.new(&2, U32)
  m = Map.set(&2, U32, m, "item-count", 5)
  take_value(Map.get(U32, 0, m, "item-count"))
```

Listing 9.2: Map の所有権付きアクセス（25_map_basics.bend）

ハンズオン／確認問題

(1) `Nat` と `U32` が自動変換されない例を探してみましょう。(2) `U32` の AND・OR・XOR と比較の違いを小さい入力で表にまとめてみましょう。(3) 数値を文字列へ変換するときに使用できる関数を調べてみましょう。

Chapter 10

モジュールと複数ファイルのプログラム

一つのファイルが長くなったら、機能ごとに分割します。**モジュール（module）**とは、関数や型をひとまとめにした再利用単位です。

10.1 `import ./... as M`

ソースを複数のファイルに分けると、数値計算、入出力、テストなどの責務を整理しやすくなります。各.bend ファイルがモジュールです。`import ./file.bend as M` は、取り込んだ定義を `M` という名前で区別できるようにします。以下の例では `module_math.bend` を取り込みます。

```
import Base

# An independent module for basic arithmetic.
def square(+x: U32) -> U32:
  (x * x : U32)

def add(x: U32, y: U32) -> U32:
  (x + y : U32)
```

Listing 10.1: 取り込まれる側の定義（module_math.bend）

```
import Base
import ./module_math.bend as Math

def main() -> U32:
  Math.square(Math.add(1, 2))
```

Listing 10.2: 別ファイルから定義を呼ぶ（19_module_main.bend）

```
bend src/19_module_main.bend
# 例: math.bend, parser.bend, main.bend
```

Listing 10.3: モジュールの実行と将来の分割

10.2 パッケージ配布と名前の扱い

BendHub には内容ハッシュ指定や名前とバージョンによる import と、`--publish` があります。公開は外部送信で取り消しの難しい操作なので、教材を GitHub に置くことと **BendHub に**

publish することは分けて判断します。ライセンスや API バージョン管理も重要です。

ハンズオン／確認問題

`square` だけを別の `.bend` へ切り出し、`import` を通して呼べることを確認してみましょう。また、ファイル同士が相互に取り込まない構成を考えてみましょう。

PART II

ライフゲームを設計して組み立てる

この Part の目次

11 ライフゲームのしくみと制作するもの 22
12 一次元配列で 8×8 盤面を表す 24
13 グライダーを初期配置する 26
14 トーラスの座標と 8 近傍 28
15 全セルを同期的に更新する 31
16 複数世代の実行とターミナル描画 35
17 テスト設計と失敗の切り分け 39
18 完成版の設計をレビューする 41

要点

この Part では、まず**ライフゲームの規則と完成条件**を整理します。その後、Part I で学んだ関数・配列・場合分けを使って、初期配置、近傍計算、世代更新、表示、テストを順に実装します。**ここで選ぶ盤面の大きさと規則は作品の仕様であり、Bend 2 という言語の制約ではありません。**

Chapter 11

ライフゲームのしくみと制作するもの

本章では、ライフゲームの規則を日本語と数式で整理します。**仕様（specification）**とは「完成したプログラムが何をするべきか」を定めた約束です。まず仕様を決めると、後で正しさを確認しやすくなります。

11.1　何を作るか

ライフゲームは二次元格子の各セルが生存（1）か死亡（0）かを持つ**セル・オートマトン（cellular automaton）**です。周囲 8 方向にある**近傍セル（neighboring cell）**の生存数を数えて、次の状態を決めます。簡単な規則ですが、移動する模様や形を変え続ける模様が生まれます。そのため、小さなプログラムでも**モデル化（modeling）**、境界での処理、テスト、性能評価を学べます。

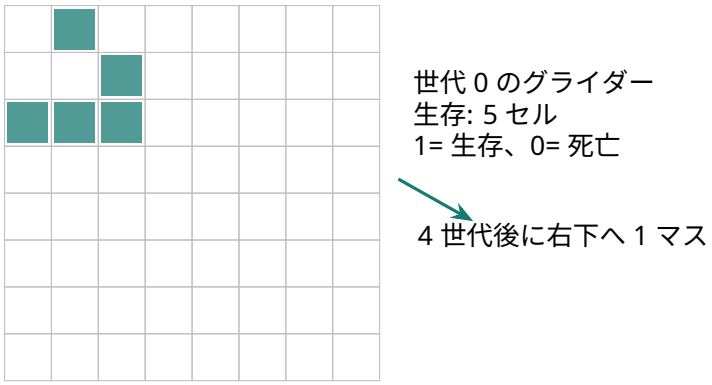
本書では格子を 8 行 ×8 列、セルを `U32` の 0 と 1、座標を (x, y)（左上が $(0, 0)$）とします。盤面の右端は左端に、下端は上端につながります。これを**周期境界条件（periodic boundary condition）**と呼びます。全セルを**同時に更新**します。

11.2　生死規則 B3/S23

セル $a \in \{0, 1\}$ と、その 8 近傍の生存数 $n \in \{0, \dots, 8\}$ に対して

$$F(a, n) = \begin{cases} 1 & n = 3 \quad \text{（生死によらず誕生／生存）} \\ a & n = 2 \quad \text{（現状維持）} \\ 0 & \text{その他} \end{cases}$$

これが **B3/S23**（Birth at 3, Survive at 2 or 3）です。いわゆる「生存セルは 2〜3 近傍で生き残る」という説明と矛盾しません。生存数だけでは次状態は決まらず、$n = 2$ なら現在の状態を参照することに注意してください。



11.3　受け入れ条件

1. 正しいグライダーを初期状態として 5 枚（世代 0〜4）の 8×8 盤面を表示します。
2. 近傍として数えるのは周囲の 8 マス**だけ**です。中心のセル自身は数えません。
3. すべてのセルの新状態を、更新前の盤面だけから計算します。**その場更新（in-place update）**を 1 セルずつ行う方法は採用しません。
4. ブロックが静止、ブリンカーが周期 2、グライダーが 4 世代で斜めへ 1 マス動くことを Python の独立実装と照合します。
5. 表示と更新を分離して、後で表示形式・実行先を交換できる形にします。

ハンズオン／確認問題

上の規則から、現在 0/1 の各場合について近傍数 0〜8 の真理値表（2×9 個）を作ってみましょう。なぜ「3 なら 1、2 なら現状、それ以外 0」の 3 分岐で全部を処理できるのかを説明してみましょう。

Chapter 12

一次元配列で 8×8 盤面を表す

前章で決めたライフゲームの規則を関数にし、続いて二次元の盤面を一次元の配列に並べます。ここから実際のシミュレータを組み立てます。

12.1　生死規則を関数で表す

前章で定めた **B3/S23** は、この作品で採用するライフゲームの規則です。一方、条件分岐を表す `match` は Bend 2 の文法です。**ゲームの規則とプログラミング言語の仕様は別物**であることを意識しましょう。

```
import Base

# The computed comparisons are matched by this helper's parameters.
def choose(is_three: Bool, is_two: Bool, alive: U32) -> U32:
  match is_three:
    case True{}:
      1
    case False{}:
      match is_two:
        case True{}:
          alive
        case False{}:
          0

# 3 neighbors => live; 2 => retain; otherwise dead.
def rule(alive: U32, +neighbors: U32) -> U32:
  choose(U32.is_eq(neighbors, 3), U32.is_eq(neighbors, 2), alive)

def main() -> U32:
  # dead cell with exactly three neighbors is born
  rule(0, 3)
```

Listing 12.1: ライフゲームの生死規則（03_rule.bend）

`rule(alive,neighbors)` では、`neighbors` が 3 なら 1、2 なら `alive`、それ以外なら 0 を返します。`choose` の内部では、Part I で学んだ `match` を使っています。`+neighbors` は同じ数値を複数回比較するための使用量指定です。生死規則は前章で決めた**作品固有の仕様**であり、Bend 2 の構文がこの規則を要求しているわけではありません。

ハンズオン／確認問題

`rule(0,3)`、`rule(1,2)`、`rule(1,4)` の返り値を確認してみましょう。現在の状態 0/1 と近傍数 0〜8 の全 18 通りを表にして、期待値と一致するかを調べてみましょう。

12.2　二次元の盤面を一次元配列に並べる

ボードを二重配列にするのではなく、長さ 64 の一次元 `Array<U32>` とします。変換は $i = 8y + x$、逆変換は $x = i \bmod 8$, $y = \lfloor i/8 \rfloor$。範囲は $0 \le x, y < 8$、$0 \le i < 64$ です。配列アクセスに用いる `U32` 添字は固定幅として扱います。

要点

配列は 64 要素に固定されていません。本書の 64 要素は 8 行 ×8 列の盤面に合わせた設計上の選択です。Bend 2 の基本的な配列は要素数が 2 の累乗になる平衡二分木で表されます。たとえば `[0 : U32*256n]` は 256 要素、`Array.new(U32, 8n, 0)` も深さ 8 の 256 要素です。`Array.new` の深さは要素数そのものではなく、要素数 2^d の指数 d です。また `Array<Array<U32>>` のように**配列を要素に持つ配列**で二次元構造を表せます。ただし、本書では座標計算と更新を見通しよくするために一次元に並べます。縦横の長さが 2 の累乗でない盤面は、余白を設けたり、別の構造を選んだりする設計が必要です。

12.3　すべてのセルを 0 で初期化する

`[0 : U32*64n]` で要素 64 個を確保します。書き込みは `Array.set(U32,b,index,value)` で新しい所有者に移る。よって `b0,b1,b2,...` と変数名を変えていくとアフィン性が見えます。インデックスは $8y + x$ で求めます。

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

添字：上段が 0〜7、次が 8〜15…

12.4　配列の添字と盤面の座標を区別する

Bend の配列は範囲外インデックスがラップする仕様を持ちます。しかし**一次元のラップ**は**二次元のトーラス隣接**と同じではありません。$(7, 0)$ の右隣は $(0, 0)$ であって $(0, 1)$ ではありません。近傍計算は座標ごとに x と y を循環させてから一次元化します。

ハンズオン／確認問題

添字 7 と 8 の意味を説明してみましょう。$(7, 0)$ から右に動いたとき `i+1` してはいけない理由を図にして考えてみましょう。

Chapter 13

グライダーを初期配置する

次に、最初の盤面を用意します。小さな生存セルの模様を指定し、正しい位置に置けたかを自分で確認します。

13.1　小さなパターンを正確に配置する

グライダーは生存セル 5 個で右下へ移動する周期 4 の模様。本書の初期配置は次の 5 点です。

$$\{(1,0),(2,1),(0,2),(1,2),(2,2)\}$$

行優先添字に変換すると $1, 10, 16, 17, 18$ になります。

```
import Base

# 8x8, row major; alive at (1,0),(2,1),(0,2),(1,2),(2,2)
def glider() -> Array<U32>:
  b0 = [0 : U32*64n]
  b1 = Array.set(U32, b0,  1, 1)
  b2 = Array.set(U32, b1, 10, 1)
  b3 = Array.set(U32, b2, 16, 1)
  b4 = Array.set(U32, b3, 17, 1)
  Array.set(U32, b4, 18, 1)

def read_value(r: Array<U32> & U32) -> U32:
  (b, v) = r
  v

def main() -> U32:
  read_value(Array.get(U32, glider(), 18))
```

Listing 13.1: 初期グライダー（05_glider.bend）

図で見れば次の配置です。`1` を生存、`0` を死亡とします。

```
01000000
00100000
11100000
00000000
00000000
00000000
00000000
00000000
```

Listing 13.2: 世代 0 の配置（8 列 ×8 行）

13.2 盤面を数える

グライダーの生存数は 5。今はセル値を `U32` として持つため、`2` や `42` も表現自体はできます。**セルが 0/1 であるという不変条件**は今のところシードと更新関数がそれ以外を作らないという開発規約に依存しています。後半では「`U32` で表現可能」と「型で 0/1 に限定される」を分けて考えます。

ハンズオン／確認問題

ブロック（$(3,3),(4,3),(3,4),(4,4)$）を初期化する `block()` を作ってみましょう。ブリンカーを $y=3, x=3,4,5$ に置く `blinker()` を作ってみましょう。出力で指定した場所だけが `1` となることを確認してみましょう。

Chapter 14

トーラスの座標と 8 近傍

どのセルが隣かを決めます。今回は画面の右端と左端、上端と下端がつながる**トーラス（torus）**状の盤面です。

14.1　盤面の端を越えた座標を計算する

左隣は $(x-1+8) \bmod 8$、右隣は $(x+1) \bmod 8$。$x \in \{0,\dots,7\}$ に限れば $(x+7) \bmod 8$ で代用できます。同様に上下も $(y+7) \bmod 8$、$(y+1) \bmod 8$ で求まる。負の数を `U32` に入れない。

$$L(x) = (x+7) \bmod 8, \quad R(x) = (x+1) \bmod 8, T(y) = (y+7) \bmod 8, \quad B(y) = (y+1) \bmod 8.$$

14.2　周囲 8 セルを読み取り生存数を数える

```
import Base

# Read one cell from an 8x8 torus.
# Each read consumes a board and produces a new board plus a cell value.
def at(b: Array<U32>, x: U32, y: U32) -> Array<U32> & U32:
  Array.get(U32, b, (y * 8 + x : U32))

# Returned board, neighbour count and original cell value.
def near_9(r: Array<U32> & U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, alive) = r
  (b, (sum, alive))

def near_8(r: Array<U32> & U32, x: U32, y: U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, v) = r
  near_9(at(b, x, y), (sum + v : U32))

def near_7(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_8(at(b, xr, yb), x, y, (sum + v : U32))

def near_6(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_7(at(b, x, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_5(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
```

```
  (b, v) = r
  near_6(at(b, xl, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_4(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_5(at(b, xr, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_3(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_4(at(b, xl, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_2(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_3(at(b, xr, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_1(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_2(at(b, x, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def neighbors(+x: U32, +y: U32, b: Array<U32>) -> Array<U32> & (U32 & U32):
  +xl = ((x + 7 : U32) % 8 : U32)
  +xr = ((x + 1 : U32) % 8 : U32)
  +yt = ((y + 7 : U32) % 8 : U32)
  +yb = ((y + 1 : U32) % 8 : U32)
  near_1(at(b, xl, yt), x, y, xl, xr, yt, yb, 0)

def blinker() -> Array<U32>:
  b0 = [0 : U32*64n]
  b1 = Array.set(U32, b0, 27, 1)
  b2 = Array.set(U32, b1, 28, 1)
  Array.set(U32, b2, 29, 1)

def count_from_pair(r: Array<U32> & (U32 & U32)) -> U32:
  (board, values) = r
  (n, alive) = values
  n

def main() -> U32:
  count_from_pair(neighbors(4, 4, blinker()))
```

Listing 14.1: 近傍数を足す（06_neighbors.bend）

`neighbors` は `(board,sum)` を返します。元の配列を `b1,b2,...,b8` へ順に渡すので、8 回の読み取りで `board` を違法に共有しません。`+xl`、`+xr`、`+yt`、`+yb` は、計算した座標値を複数の読み取りに使うために必要。

要点

近傍の重複を防ぐには、3×3 の中心 (x, y) を除く 8 箇所を列挙し、同じ (x, y) を含めていないか紙に書いて点検します。**旧盤面から読み、読んだ配列を戻す**という型付きの設計は、この過程の所有権をはっきりさせる。

14.3 境界にいるセルを手計算する

例えば $(0,0)$ の 8 近傍は $(7,7),(0,7),(1,7),(7,0),(1,0),(7,1),(0,1),(1,1)$。四隅のセルで座標の循環をテストします。$(4,4)$ など内部座標だけを調べても境界バグは見つからない。

ハンズオン／確認問題

(1) $(0,0)$ の 8 近傍座標を書き出してみましょう。(2) 初期グライダーに対して $(1,1)$ の生存近傍数を手計算してみましょう。(3) コード内で `+xl` を外して、どの使用が重複するか確認してみましょう。(4) 06_neighbors.bendで `neighbors(4,4,...)` の期待値は 3 になることを考えてみましょう。

Chapter 15

全セルを同期的に更新する

いよいよ 1 世代の更新を実装します。**同期更新（synchronous update）**とは、次の盤面を計算するとき、すべてのセルが同じ古い盤面を読む方法です。

15.1　更新に必ず二枚の盤面を使う

正しい更新では、時刻 t の状態 B_t だけを見て B_{t+1} を作る。例えば上から順に古い配列に直接書き込むと、後で処理するセルが更新済みの近傍値を読みます。その瞬間、同時更新ではなく走査順依存の別のゲームになります。

$$B_{t+1}(x,y) = F\Big(B_t(x,y), \sum_{(u,v)\in N(x,y)} B_t(u,v)\Big).$$

15.2　再帰処理が終わる条件を決める

燃料（fuel）とは、再帰を続ける残り回数を表す値を比喩的に呼んだものです。燃料が 0 になれば止まる、と決めれば無限に繰り返さないことを説明しやすくなります。

`fill(todo, i, old, out)` の `todo` は `Nat` で、残り何セル更新するかを表します。`match todo` で `0` なら出力盤面を返し、`1n+p` なら 1 セル更新して `fill(p,...)` を呼びます。再帰が必ず止まることを `todo` の減少から説明できます。

```
import Base

# Fixed 8x8 toroidal Game of Life, B3/S23.
# Cells are U32 values, maintained in {0,1} by seed and rule.
def choose(is_three: Bool, is_two: Bool, alive: U32) -> U32:
  match is_three:
    case True{}:
      1
    case False{}:
      match is_two:
        case True{}:
          alive
        case False{}:
          0

def rule(alive: U32, +n: U32) -> U32:
  choose(U32.is_eq(n, 3), U32.is_eq(n, 2), alive)

```

```
# Each read consumes a board and produces a new board plus a cell value.
def at(b: Array<U32>, x: U32, y: U32) -> Array<U32> & U32:
  Array.get(U32, b, (y * 8 + x : U32))

# Returned board, neighbour count and original cell value.
def near_9(r: Array<U32> & U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, alive) = r
  (b, (sum, alive))

def near_8(r: Array<U32> & U32, x: U32, y: U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, v) = r
  near_9(at(b, x, y), (sum + v : U32))

def near_7(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_8(at(b, xr, yb), x, y, (sum + v : U32))

def near_6(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_7(at(b, x, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_5(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_6(at(b, xl, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_4(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_5(at(b, xr, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_3(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_4(at(b, xl, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_2(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_3(at(b, xr, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_1(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_2(at(b, x, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def neighbors(+x: U32, +y: U32, b: Array<U32>) -> Array<U32> & (U32 & U32):
  +xl = ((x + 7 : U32) % 8 : U32)
  +xr = ((x + 1 : U32) % 8 : U32)
  +yt = ((y + 7 : U32) % 8 : U32)
  +yb = ((y + 1 : U32) % 8 : U32)
  near_1(at(b, xl, yt), x, y, xl, xr, yt, yb, 0)

def glider() -> Array<U32>:
  b0 = [0 : U32*64n]
  b1 = Array.set(U32, b0,  1, 1)
  b2 = Array.set(U32, b1, 10, 1)
  b3 = Array.set(U32, b2, 16, 1)
  b4 = Array.set(U32, b3, 17, 1)
  Array.set(U32, b4, 18, 1)

# Decreasing Nat argument first: accepted recursive shape.
```

```
# Carry a board, neighbour count, and original cell state together.
# Destructuring is allowed here because r is a parameter.
def fill(todo: Nat, +i: U32, r: Array<U32> & (U32 & U32), out: Array<U32>) -> Array<U32>:
  match todo:
    case 0n:
      out
    case 1n+p:
      (old, pair) = r
      (n, alive) = pair
      next = rule(alive, n)
      out1 = Array.set(U32, out, i, next)
      +j = (i + 1 : U32)
      x = (j % 8 : U32)
      y = ((j / 8 : U32) % 8 : U32)
      fill(p, j, neighbors(x, y, old), out1)

def step(board: Array<U32>) -> Array<U32>:
  fill(64n, 0, neighbors(0, 0, board), [0 : U32*64n])

def evolve(steps: Nat, board: Array<U32>) -> Array<U32>:
  match steps:
    case 0n:
      board
    case 1n+p:
      evolve(p, step(board))

# Reading consumes and returns the array, which can be dropped here.
def value_from_pair(r: Array<U32> & U32) -> U32:
  (b, v) = r
  v

def main() -> U32:
  value_from_pair(Array.get(U32, evolve(4n, glider()), 26))
```

Listing 15.1: 完成した純粋な更新核（07_life_core.bend）

15.3 ソースを読む順番

1. `glider()` は 5 セルを 1 にした入力の配列を作る。
2. `step(board)` は旧盤面と全 0 の新盤面を用意します。
3. `fill(64n,0,old,out)` はセル $i = 0$ から 63 を処理します。
4. `neighbors` は旧盤面を読み、`Array.get` は対象セルの現在値を読みます。
5. `rule` で次状態を決め、`Array.set` で新盤面の対応位置に保存します。
6. `fill` が残りを処理します。旧盤面は最後まで変更しません。

この走査は計算量 $O(WH)$、追加記憶量 $O(WH)$ です。8 方向参照の定数が大きいため、厳密な性能比較では「セル 1 個当たりの読み込み・格納」を数える。

15.4 別の実装と結果を比較する

同じ `neighbors` を使ってテスト期待値を作ると、その関数に同じ誤りがあった場合に見逃します。したがってscripts/reference.pyは、別の記述・別の実装言語で定義した生存セルの `set` から計算します。**2 つが一致しても全入力の正しさを証明したわけではない**が、共通の実装ミスは見つけやすい。

ハンズオン／確認問題

(1) `fill` の先頭引数を `U32` のカウンタに置き換えると停止性検査にどんな障害があるか。(2) `step` を旧配列 1 枚への更新に変え、グライダーの軌跡がどこから破綻するか観察してみましょう。(3) `step(glider())` の 8×8 値をオラクルと照合してみましょう。

Chapter 16

複数世代の実行とターミナル描画

1 世代だけでなく複数世代を動かし、ターミナルへ表示します。計算と表示を別々の関数にすると、後から見た目だけを変更できます。

16.1　世代を繰り返す関数

evolve(steps,board) は、0 ステップならそのまま返し、正の Nat なら step を呼んでから evolve(p,...) を呼びます。ここで p が 1 減った残りの自然数になっています。型検査は停止の根拠を要求するため、evolve の再帰が無限にループする例は通常許されません。

16.2　出力を作る関数は更新と分ける

一つの行は 8 セルを読み文字列化し、8 行を結合します。render_cells と render_rows は (board,String) を返します。読み出しに使った配列を次の処理へ回せるようにするためです。IO.print は show だけが持つ。

```
import Base

# Same algorithm as 07_life_core, now with a text renderer.
def choose(is_three: Bool, is_two: Bool, alive: U32) -> U32:
  match is_three:
    case True{}:
      1
    case False{}:
      match is_two:
        case True{}:
          alive
        case False{}:
          0

def rule(alive: U32, +n: U32) -> U32:
  choose(U32.is_eq(n, 3), U32.is_eq(n, 2), alive)

# Each read consumes a board and produces a new board plus a cell value.
def at(b: Array<U32>, x: U32, y: U32) -> Array<U32> & U32:
  Array.get(U32, b, (y * 8 + x : U32))

# Returned board, neighbour count and original cell value.
def near_9(r: Array<U32> & U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, alive) = r
```

```
  (b, (sum, alive))

def near_8(r: Array<U32> & U32, x: U32, y: U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, v) = r
  near_9(at(b, x, y), (sum + v : U32))

def near_7(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_8(at(b, xr, yb), x, y, (sum + v : U32))

def near_6(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_7(at(b, x, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_5(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_6(at(b, xl, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_4(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_5(at(b, xr, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_3(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_4(at(b, xl, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_2(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_3(at(b, xr, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_1(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_2(at(b, x, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def neighbors(+x: U32, +y: U32, b: Array<U32>) -> Array<U32> & (U32 & U32):
  +xl = ((x + 7 : U32) % 8 : U32)
  +xr = ((x + 1 : U32) % 8 : U32)
  +yt = ((y + 7 : U32) % 8 : U32)
  +yb = ((y + 1 : U32) % 8 : U32)
  near_1(at(b, xl, yt), x, y, xl, xr, yt, yb, 0)

def glider() -> Array<U32>:
  b0 = [0 : U32*64n]
  b1 = Array.set(U32, b0,  1, 1)
  b2 = Array.set(U32, b1, 10, 1)
  b3 = Array.set(U32, b2, 16, 1)
  b4 = Array.set(U32, b3, 17, 1)
  Array.set(U32, b4, 18, 1)

# Carry a board, neighbour count, and original cell state together.
# Destructuring is allowed here because r is a parameter.
def fill(todo: Nat, +i: U32, r: Array<U32> & (U32 & U32), out: Array<U32>) -> Array<U32>:
  match todo:
    case 0n:
      out
    case 1n+p:
      (old, pair) = r
```

```
      (n, alive) = pair
      next = rule(alive, n)
      out1 = Array.set(U32, out, i, next)
      +j = (i + 1 : U32)
      x = (j % 8 : U32)
      y = ((j / 8 : U32) % 8 : U32)
      fill(p, j, neighbors(x, y, old), out1)

def step(board: Array<U32>) -> Array<U32>:
  fill(64n, 0, neighbors(0, 0, board), [0 : U32*64n])

# The recursion itself receives the array and the value as a parameter.
def read_cells(todo: Nat, +i: U32, text: String, r: Array<U32> & U32) -> Array<U32> & String:
  match todo:
    case 0n:
      (board, v) = r
      (board, text ++ U32.show(v))
    case 1n+p:
      (board, v) = r
      newer = text ++ U32.show(v)
      read_cells(p, (i + 1 : U32), newer, Array.get(U32, board, (i + 1 : U32)))

def render_cells(n: Nat, +i: U32, board: Array<U32>) -> Array<U32> & String:
  match n:
    case 0n:
      (board, "")
    case 1n+p:
      read_cells(p, i, "", Array.get(U32, board, i))

def read_rows(todo: Nat, +i: U32, text: String, r: Array<U32> & String) -> Array<U32> & String:
  match todo:
    case 0n:
      (board, line) = r
      (board, text ++ line ++ "\n")
    case 1n+p:
      (board, line) = r
      newer = text ++ line ++ "\n"
      read_rows(p, (i + 8 : U32), newer, render_cells(8n, (i + 8 : U32), board))

def render_rows(n: Nat, +i: U32, board: Array<U32>) -> Array<U32> & String:
  match n:
    case 0n:
      (board, "")
    case 1n+p:
      read_rows(p, i, "", render_cells(8n, i, board))

def show_rendered(todo: Nat, +generation: U32, r: Array<U32> & String) -> IO(Unit):
  match todo:
    case 0n:
      (board, picture) = r
      do IO<Unit>:
        IO.print("generation " ++ U32.show(generation))
        IO.print(picture)
        IO.print("done")
    case 1n+p:
      (board, picture) = r
      do IO<Unit>:
        IO.print("generation " ++ U32.show(generation))
        IO.print(picture)
        show_rendered(p, (generation + 1 : U32), render_rows(8n, 0, step(board)))

def main() -> IO(Unit):
  show_rendered(4n, 0, render_rows(8n, 0, glider()))
```

Listing 16.1: 完成版：5 世代を表示するシミュレータ（08_life_terminal.bend）

16.3 実行する

```
bend src/08_life_terminal.bend --check-only
bend src/08_life_terminal.bend
bend src/08_life_terminal.bend -o build/life
./build/life --threads 1
```

Listing 16.2: 完成版の実行

このプログラムは**世代 0〜4 の 5 枚**を表示します。`IO.print` が末尾の改行を追加するので、盤面の間には空行が入る。`show(5n,0,glider())` を `show(10n,0,glider())` にすれば 10 枚になります。`U32` の世代番号が巨大になるほどの実験は、この入門版の想定外。

16.4 期待される世代 4

```
00100000
00010000
01110000
```

Listing 16.3: Python オラクルが算出する世代 4（Bend 未実行）

これは世代 0 のグライダーが右下に 1 だけ移動した形を**一部抜粋**したもの（上段は y=1、続く行が y=2,3）。完全な 8 行はscripts/reference.pyで出す。

ハンズオン／確認問題

(1) `show` の世代数を 3、5、10 に変更してみましょう。(2) 表示を `1/0` から `#/･` へ変えるための `glyph` 関数を追加してみましょう。(3) 初期配置の章で作った `block()` を使い、静止パターンであることを調べてみましょう。

Chapter 17

テスト設計と失敗の切り分け

要点

テストが失敗したら非ゼロ終了します。 CI（継続的インテグレーション）は、コードを GitHub へ送ったときに自動で検査する仕組みです。本書の Python テストは `assertEqual` の不一致で失敗し、Bend の実行テストも終了コードと盤面を検査します。単に `PASS` と印字するだけの検査にはしません。

動作を確認するための**テスト（test）**を作ります。完成したプログラムと、別方式で作った参照実装の結果を比べます。

17.1　プログラムの正しさをテストで確認する

ここまででコードを組み立てました。しかし「文法が正しい」ことと「ライフゲームの規則を正しく実装した」ことは別です。期待する盤面をあらかじめ用意し、結果を照合して確認します。

コンパイラが型の整合性と停止性を確認しても、`neighbors` で中央セルを足してしまう誤りや、`rule` で 2 と 3 を取り違える誤りは型だけでは防げない。少なくとも**局所規則**、**初期配置**、**時刻発展**、**境界条件**の 4 レベルで検査します。

17.2　参照実装を動かす

```
python3 -m unittest discover -s tests -v
python3 scripts/reference.py
python3 scripts/check_bend.py
python3 scripts/check_bend.py --native
```

Listing 17.1: 付属の Python 独立テスト（標準ライブラリのみ）

最後の 2 行は Bend が導入されている場合の任意の**統合テスト**です。`check_bend.py` は Bend の表示から 5 枚の盤面を取り出し、Python オラクルの出力と改行込みで比較します。失敗したら diff を表示します。

17.3 用意された独立テスト 6 件

テスト	検査内容
`test_empty_stays_empty`	空盤面は空のまま。
`test_block_is_still_life`	ブロックは 1 世代後も変化しません。
`test_blinker_oscillates`	ブリンカーは横 → 縦 → 横の 2 周期。
`test_glider_moves...`	4 世代後、グライダーが斜めに 1 進みます。
`test_alive_count...`	グライダーの生存数は 12 世代を通じて 5。
`test_transcript...`	出力の枚数・終端テキストの形。

注意

テストの種類を区別します。**Python の独立テスト 6 件**と、**Bend 2.0.37 での統合テスト**は CI で成功しました。後者では 5 世代全セルの出力一致を確かめています。ただし、異なる Bend のバージョンや GPU 実行でも同じとは限りません。

ハンズオン／確認問題

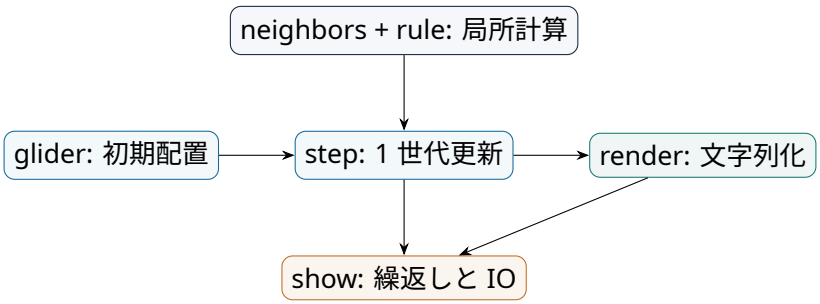
(1) トーラス端をまたぐブリンカーを設定し、2 周期を検証するテストを書いてみましょう。(2) `rule` の「2 のとき現状維持」を意図的に崩し、どのテストが落ちるかを報告してみましょう。(3) 手計算・Python・Bend の 3 列からなる検証表を作ってみましょう。

Chapter 18

完成版の設計をレビューする

いったん完成したプログラムを読み直します。**保守性（maintainability）**とは、後から機能追加や修正をしやすいことです。

18.1　関数が担う責務



`rule` は 1 セルの遷移、`neighbors` は 8 セルの集計、`step` は盤面全体の遷移、`render` は表示、`show` は実行制御を担当します。更新ロジックと IO を分けることは、後で GUI やファイル出力に置き換える際に重要です。

18.2　保守性チェック

1. **定数が散らばっている**: 8 と 64 が複数箇所にある。将来は専用の盤面型に集約します。
2. **0/1 の意味が型に表れていない**: `U32` から `Cell` 型に移行する余地があります。
3. **表示が計算重視ではない**: 文字列連結は小規模ならよいが、大規模なら行単位のバッファにします。
4. **仕様が固定**: 出力枚数と初期配置をコードで編集します。コマンドライン引数は発展課題。
5. **多コア利用がまだない**: 次部で用途に合った表現と分割法を検討します。

ハンズオン／確認問題

関数ごとに「入力型・出力型・副作用の有無・計算量」を表にしましょう。どの関数なら GPU に移行しても意味が変わらず、どの関数が CPU に残るべきかを考察してみましょう。

PART III

アプリケーション基礎を広げる

この Part の目次

19 モナドと do 構文を理解する 43
20 画面出力とファイル操作を扱う 46
21 複数の入出力処理を進める方法 48
22 ライフゲームを画面に表示する準備 50
23 Bend のプログラムを別の環境で動かす 52

Chapter 19

モナドと do 構文を理解する

この章では、手続き型言語の「一行ずつ処理する」書き方と、関数型言語で**計算の文脈をつなぐ**書き方の違いを学びます。特に、失敗するかもしれない数値変換を例に**モナド（monad）**を理解します。

19.1　値を返す計算と、失敗し得る計算の違い

`(2 + 3 : U32)` は必ず `U32` を返す計算です。一方、文字列を整数に変える `U32.read("42")` は成功するとは限りません。`"abc"` を渡した場合、返すべき整数がありません。**Maybe** はこの違いを型で示します。成功なら `Some{42}`、失敗なら `None{}` です。`Maybe<&2, U32>` という型は、**U32 が得られるかもしれない計算結果**を表します。先頭の `&2` は値の再利用可能性に関する Bend 2 固有の量指定です。ここでは「値があるかないか」と別の役割であることを押さえてください。

19.2　手続き型の失敗判定を Maybe に置き換える

従来の言語では、整数を読み取った後に成否を調べ、失敗ならそこで処理を止める `if` 文を何度も書くことがあります。`Maybe` なら**失敗の伝播（failure propagation）**を共通の仕組みに任せられます。

```
import Base

# A failed read (None) short-circuits the Maybe block.
def add_strs(a: String, b: String) -> Maybe<&2, U32>:
  do Maybe<&2, U32>:
    x : U32 <- U32.read(a)
    y : U32 <- U32.read(b)
    return (x + y : U32)

def main() -> Maybe<&2, U32>:
  add_strs("40", "2")
```

Listing 19.1: 二つの文字列を安全に加算する（21_monadic_maybe.bend）

実行例は `add_strs("40", "2")` が `Some{42}`、`add_strs("abc", "2")` が `None{}` です。**最初の解析に失敗した場合、次の解析と加算へ進みません**。失敗を表すために 0 や負数などの「特別な数値」を予約する必要もありません。

19.3 モナドとは何を共通化した仕組みか

モナドは、値を包む型の仕組みと、**その型を保ったまま複数の計算をつなぐ規則**を組にした抽象化です。Bend 2 では、例えば Maybe に次の二つの操作があります。

操作	役割
M.pure	普通の値を M の計算結果として包みます。Maybe なら Some{x} に相当します。
M.bind	包まれた値を次の関数へ渡し、次の計算も M の形で返します。Maybe なら None{} を次へ進めず、そのまま返します。

ここで M は Maybe や IO などの型を表す仮の名前です。「モナド＝入出力」でも「モナド＝副作用」でもありません。Maybe は失敗、Result は失敗の理由、IO は外部とのやり取りという**異なる文脈**を扱います。

19.4 bind を直接書いてみる

do は新しい種類の計算を実行する命令ではなく、bind と pure を読みやすく書くための**構文糖（syntactic sugar）**です。次のコードでは、解析に失敗したときの処理を Maybe.bind へ任せています。

```
import Base

# The do notation in 21_monadic_maybe.bend expands to binds and pure.
def add_strs_explicit(a: String, b: String) -> Maybe<&2, U32>:
  Maybe.bind(&2, U32, U32, U32.read(a), x =>
    Maybe.bind(&2, U32, U32, U32.read(b), y =>
      Maybe.pure(&2, U32, (x + y : U32))))

def main() -> Maybe<&2, U32>:
  add_strs_explicit("40", "2")
```

Listing 19.2: Maybe.bind を直接使う（30_monad_explicit.bend）

このプログラムと、前節の do Maybe<&2, U32>: を使ったプログラムは同じ結果を返す設計です。x : U32 <- U32.read(a) は「成功した値を x へ受け渡して続行する」という意味であり、**通常の変数代入（x = ...）とは違います**。

要点

do M<..., R>: の x : A <- m は、M.bind(..., A, R, m, x => ...) を使う形へ展開されます。return e は M.pure(..., R, e) を使う形へ展開されます。ここでの return は、C 言語の return のような「命令的な早期脱出」と混同しないでください。

19.5 モナドの規則と Result への発展

一般にモナドとして使う pure と bind は、計算を一貫してつなぐために**左単位律・右単位律・結合律**と呼ばれる規則を満たすように設計します。初めは数式を暗記する必要はありません。「普通の値を包んでから続ける操作」と「そのまま続ける操作」が同じ結果になること、つなぐ場所の括り方を変えても意味が変わらないことが要点です。なお、**pure と bind という名前だけを持つ型が、必ずこの規則を満たすわけではありません**。規則が成り立つかは実装側の責任です。Result

は Done{value} と Fail{error} を使い、失敗の理由を残せます。Maybe は単に失敗したことだけを知らせればよいとき、Result は**何が間違っていたかを説明したいとき**に向いています。

19.6 同じ do で入出力をつなぐ

以降の章では do IO<Unit>: を使います。IO.print は「文字列を表示するという**効果（effect）**」を記述します。do 内に順番に並べることで、表示の順序を明示できます。**Maybe では失敗を、IO では効果の順序を扱いますが、つなぎ方の構文は共通**です。この違いを確認してからファイル操作に進みましょう。

ハンズオン／確認問題

add_strs("12", "30") と add_strs("abc", "30") の結果を予想し、実行してみましょう。do 版と bind 版の結果を比較してみましょう。さらに、三つの文字列を足す関数を作り、途中で失敗した場合の返り値を確かめてみましょう。

Chapter 20

画面出力とファイル操作を扱う

プログラムがファイルや画面へ作用する場面を扱います。Bend では、入出力（I/O: input/output）を型で明示し、純粋な計算と区別します。

20.1　値の計算と効果の実行

ライフゲームの世代更新は純粋関数で、表示は `IO` で行います。`do IO<R>:` の `<-` は**前の効果が返した値を取り出し、次の効果へ渡す**構文です。`return` はこの文脈では普通の関数を途中で抜ける命令ではなく、モナドの `pure` に相当します。

```
import Base

# Concurrent effects are different from pure parallel CPU calls.
def delayed_message(number: U32) -> IO(String):
  do IO<String>:
    IO.sleep(10)
    return "result = " ++ U32.show(number)

def main() -> IO(Unit):
  do IO<Unit>:
    channel : Chan(String) <- IO.fork(String, delayed_message(42))
    IO.print("waiting for another computation")
    message : String <- IO.join(String, channel)
    IO.print(message)
```

Listing 20.1: 非同期効果を含む IO プログラム（20_io_fork.bend）

20.2　入力・引数・ファイル・失敗

`IO.args` は実行時引数を受け取る。`IO.get_env` は環境変数の読み出し、`IO.try` は失敗し得る IO を扱います。ファイル API は `File.open/read/write/close` 等の実際の定義を `bend base File` で調査します。**ファイルハンドルはアフィン**なので、使ったハンドルを適切に返して次の操作へ渡します。失敗時の `Result` を握りつぶさない。

20.3 ファイルを実際に扱う

File.open は失敗を含む Result を返します。File.write は**次の所有権を持つ File と操作結果**を返し、ファイルを閉じずに捨てないことが重要です。次の例はlife-demo-output.txtを**上書き**します。実験用ディレクトリでのみ実行すること。

```
import Base

# WARNING: overwrites life-demo-output.txt in the working directory.
# Close the affine handle even when writing fails.
def finish(result: File & Result<&1, &1, U32 & String, Unit>) -> IO(Unit):
  match result:
    case (file, status):
      do IO<Unit>:
        File.close(file)
        IO.pass(Unit, status)
        IO.print("saved")

def main() -> IO(Unit):
  do IO<Unit>:
    file : File <- IO.try(File, File.open("life-demo-output.txt", "w"))
    result : File & Result<&1, &1, U32 & String, Unit> <- File.write(file, "generation=0")
    finish(result)
```

Listing 20.2: ファイルへ書き、失敗時にも閉じる（26_file_write.bend）

```
mkdir -p scratch && cd scratch
bend ../src/26_file_write.bend
cat life-demo-output.txt
cd ..
```

Listing 20.3: ファイルの試験（上書きするので注意）

20.4 Life での実習

完成版の固定回数表示を**コマンドラインで世代数を与える設計**へ変更します。IO.args から List<String> を受け取り、U32.read で安全に解釈し、失敗時は理由を表示します。Nat へ変換した回数だけ進める。ただし正確な Base の型は使用版で確認してから実装します。

ハンズオン／確認問題

(1) IO.print の実行順序を予想してみましょう。(2) IO.get_env が返すエラーの扱いを bend base IO.get_env で調べてみましょう。(3) 文字列解析と世代更新を別関数に分けてみましょう。

Chapter 21

複数の入出力処理を進める方法

複数の仕事を同時に進める方法は一つではありません。ここでは**並行処理（concurrency）**と**並列計算（parallel computation）**の違いを明確にします。

21.1 IO.fork と IO.join

IO.fork は効果付き計算を独立に開始し、結果が到着する Chan を返します。IO.join はその結果を待つ。これは**ファイルやネットワークの待ち時間がある処理を重ねる**ための非同期処理であり、計算を分割して CPU コアへ流す次の Part の**並列呼び出し**とは別です。

```
import Base

# Concurrent effects are different from pure parallel CPU calls.
def delayed_message(number: U32) -> IO(String):
  do IO<String>:
    IO.sleep(10)
    return "result = " ++ U32.show(number)

def main() -> IO(Unit):
  do IO<Unit>:
    channel : Chan(String) <- IO.fork(String, delayed_message(42))
    IO.print("waiting for another computation")
    message : String <- IO.join(String, channel)
    IO.print(message)
```

Listing 21.1: fork/join を実際に実行する例（再掲）

21.2 キャンセル・タイムアウト・共有状態

IO.within は期限を過ぎた操作に None を返すが、**負けた処理を自動キャンセルするとは限らない**。Chan.send/recv/close などによる待ち合わせでは、相手を失うとデッドロックになり得ます。Life ではフレーム描画とイベント受付を効果として分け、世代更新だけを純粋に保つことが安全な境界になります。

ハンズオン／確認問題

(1) `IO.sleep` を入れて二つの処理の進行を観察してみましょう。(2) CPU 上での純粋関数の並列呼び出しと、IO.fork を比較表にまとめてみましょう。(3) タイムアウト時に完了しなかった処理が残る可能性を説明してみましょう。

Chapter 22

ライフゲームを画面に表示する準備

GUI（graphical user interface）はウィンドウや画像で操作する画面です。ターミナル版の処理を生かして、画面表示へ拡張する設計を学びます。

22.1　純粋な描画とイベント更新

Bend には App.run、Image、Event などがあります。Image は Pix（一様な色）と Qua（四分木）で表せる。App は状態を画像へ変える view と、イベントから次状態を決める tick を持ちます。

```
import Base

# A minimal window demo; this is NOT the graphical Life simulator.
def tick(events: List<Event>, color: U32) -> IO(Maybe<U32>):
  match events:
    case Nil{}:
      IO.pure(Maybe<U32>, Some{color})
    case Con{Close{}, rest}:
      IO.pure(Maybe<U32>, None{})
    case Con{e, rest}:
      tick(rest, (color + 1 : U32))

def main() -> IO(Unit):
  App.run(~U32, ~App{+s => (s, Pix{s}), tick}, "Bend 2 App Preview", 256, 256, 0)
```

Listing 22.1: ウィンドウを開く最小 App（22_app_preview.bend）

22.2　Life の画面表示への発展

最小 App はライフゲーム本体ではなく**ウィンドウとイベントを学ぶ単独デモ**です。これを起点に、状態を Array<U32> と世代番号の組に変更し、view で 8×8 を四分木画像へ変換し、tick で更新・一時停止・終了を行う設計に進める。view はアフィンな状態を消費するので、**状態と画像のペアを返す**必要があります。

注意

App.run にはネイティブ環境のウィンドウシステムが必要です。Linux では X11 用ライブラリが必要な場合があり、WSL/ヘッドレス環境では起動できないことがあります。画面表示

はターミナル版の合格条件には含めない。

ハンズオン／確認問題

(1) `Pix` と `Qua` で 4 分割の画像を設計してみましょう。(2) Close イベントで `None` を返す理由を説明してみましょう。(3) Life の盤面更新と描画イベントの役割分担を図にしてみましょう。

Chapter 23

Bend のプログラムを別の環境で動かす

Bend のコードは単独で完結するとは限りません。他の言語や OS の機能と接続する仕組み、実行形式の違い、公開時の注意点を確認します。

23.1　標準 IO の外に出るとき

Base は端末、環境変数、ファイル、ネットワーク（TCP/UDP）、プロセス起動、音声・ウィンドウ等の効果を提供します。新しい効果は `import "./x.c"` と `import "./x.js"` によってホスト側で実装する方式を採る。**純粋な計算と、OS を変更する処理**を混同しません。ライフゲームならファイルの読み込み、初期配置の保存、データ形式変換などが応用例になります。

23.2　複数のターゲットを知る

```
bend src/01_hello.bend -o build/hello
bend src/19_module_main.bend -o build/module.js
bend src/19_module_main.bend -o build/module.c
bend src/19_module_main.bend -o build/module.mjs
bend guide effects
```

Listing 23.1: ターゲットを切り替えて学ぶ（対応状況は環境次第）

`.mjs` は非 IO の定義を JS から import する用途です。ブラウザは HTML を入力して`-o dist` とするバンドルの仕組みもある。JavaScript ターゲットの並列実行はネイティブの CPU/GPU 並列と同一とは限りません。`Process.run` は外部プログラムを起動するため、信頼できる引数と実行ファイルしか渡さない。

23.3　公開プロジェクトでの安全性

外部ライブラリのバージョン、サンプルコードの出所、権利情報を確認します。`--publish` は GitHub への push と別操作であり、**意図せず外部へ永続公開しない**。Bend は進化が速いため、`bend version`・`bend guide`・`bend base` を学習実験の記録に残す。

ハンズオン／確認問題

(1) C/JS 出力を生成し、コード中で元の `main` に対応する箇所を探してみましょう。(2) ネイティブ・JS・GPU の用途を比較してみましょう。(3) 既存の Life をブラウザ App へ持ち込む前に不足している 3 つの実装要素を挙げてみましょう。

PART IV

Bend の強みを掘り下げる

この Part の目次

24 再帰の停止性と計算量 55
25 アフィン性・所有権・配列の更新 56
26 CPU 並列：純粋な並列呼び出しを理解する 58
27 GPU 呼び出しと実行レーン 60
28 性能評価：速さを数字で扱う 62
29 型を使って条件を表す 64
30 規則の正しさを証明する 65
31 局所証明から盤面全体の正しさへ 67
32 上級プロジェクト：可変盤面と GPU 化を設計する 68
33 量付き Kind と汎用的な型を整理する 70
34 証明の基本操作：帰納法・書き換え・反証 72

Chapter 24

再帰の停止性と計算量

ここから応用編です。**停止性（termination）**は処理が必ず終わる性質、**計算量（complexity）**は入力サイズに対する時間や記憶量の増え方です。別々に考えましょう。

24.1　停止性は速度の話ではない

Bend では原則として再帰関数の停止が必要です。`fill(todo,...)` は `todo`$= 0$ で停止し、それ以外は $p = todo - 1$ で再帰します。`evolve(steps,...)` も同様です。これは**有限ステップで終わる**ことの根拠であり、**実行が速い**ことの保証ではありません。

24.2　再帰を安全に終わらせる引数の指定

公式ガイドには、再帰呼び出しの引数を左から見て、ある引数が元の引数の小さい部分になるまで手前の引数がそのままでなければならないという制約があります。だから `fill(todo,i,old,out)` では `todo` を先頭にします。`fill(i,todo,...)` にして `i+1` を先に渡すと認識されにくい。

24.3　更新処理の計算量を見積もる

幅 W、高さ H の盤面で、1 セルに固定 8 回の読み込みと 1 回の書き込みがあるとします。1 世代の時間は $T(W,H) = \Theta(WH)$、g 世代なら $\Theta(gWH)$。別の出力盤面のために $\Theta(WH)$ の追加領域を確保します。画面出力の文字列構築は別に考えます。

ハンズオン／確認問題

(1) $N = WH$ として 1 世代に対して `Array.get` が何回呼ばれるか。(2) 再帰の最大深さはおよそ何か。(3) `Array.get` が実際には定数時間でないと仮定した場合、$T(N)$ の結論がどう変わるか。

Chapter 25

アフィン性・所有権・配列の更新

配列がどう受け渡されるかをもう一度整理します。**所有権（ownership）**は「この値を使える責任を、どの変数が持つか」という見方です。

25.1　一度しか使わないとは何か

Bend は **affine** であり、名前に結びついた資源は高々 1 回しか消費できません。リストなど `Data` の値は `+` によって再利用できるが、配列やハンドルは通常 `Type` なのでできません。「メモリが安全か」を局所的に検査でき、更新中の別名参照を抑えられます。

25.2　複数のセルを読むときの処理

近傍 8 回の読出しで毎回 `(board,value)` を受け取る。`board` の所有権を順に渡しているだけなので、`Array.get` のたびに配列全体をコピーすることを意味しません。`Array.set` も同じ配列資源の所有を次へ引き継ぐ。これを「純粋な関数型の見た目と in-place 更新の両立」と理解します。

25.3　ひとつの盤面を並列更新する際の課題

`Array<U32>` を `+board` にして 2 つの並列関数へ渡すと、`Array` は `Data` ではないので型が許しません。**無理に共有するのではなく、処理単位が所有できる入力データの構造を設計し直す**。候補は、境界の読み取り用ハローを含む独立タイル、木構造による領域分割、あるいは GPU に適したフラットなバッファを専門の手順で受け渡す方法です。

注意

12_affine_probe_INVALID.bendのような「同じ `board` を二度読む」コードは意図的にエラーを起こす。Bend の `Array` を一般的な `const&` 参照や GC 言語の共有配列と同じだと考えない。

ハンズオン／確認問題

逐次で 8 回読み取るコードを書いた後、`(b1,v1)` の代わりに戻された配列を無視して元配列を再使用してみましょう。所有権エラーを説明し、型安全であることと並列であることが別の主張だと説明してみましょう。

Chapter 26

CPU 並列：純粋な並列呼び出しを理解する

一つの大きな計算を独立した小さな計算に分け、複数の CPU コアへ割り振ります。これが本章の**並列計算（parallel computation）**です。

26.1　Bend で並列計算を記述する

並列な二つの呼び出しは `a b = f(x) g(y)` という記法で示します。出力の `a` と `b` は両方揃ってから後続の式で使えます。競合がないことは純粋性・アフィン性によってサポートされるが、負荷が不均衡だと高速化は限定的。

```
import Base

# A genuine balanced fork/join benchmark of independent calls.
def pow2(+depth: Nat) -> U32:
  match depth:
    case 0n:
      1
    case 1n+p:
      left right = pow2(p) pow2(p)
      (left + right : U32)

def main() -> U32:
  pow2(12n)
```

Listing 26.1: 均等に分割できる並列再帰（09_parallel_tree.bend）

26.2　実行形態を分けて測る

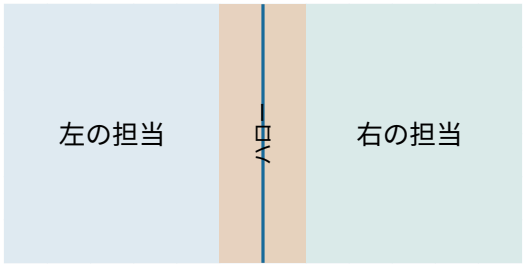
```
bend src/09_parallel_tree.bend -o build/tree
./build/tree --threads 1
./build/tree --threads 2
./build/tree --threads 4
./build/tree --threads 8
```

Listing 26.2: ネイティブ・スレッド数を変える

$2^{12} = 4096$ が一致するのを確認します。時間差を測るときは `time` 等の同じ方法で、複数回、同じマシンの同じ入力を比較します。`bend src/09_parallel_tree.bend` で得たインタプリタ相当の時間をネイティブ並列の評価に用いない。

26.3　ライフゲームへの設計移植

単一盤面を領域 L, R に分割し、各領域の新状態を独立に計算したい。ただし左右の境界近くでは相手側の旧状態が必要です。そこで **halo exchange**（境界 1 セル分のコピー）を使う設計が考えられます。適切な分割単位 b と並列タスク数 p を選び、作業量をおおよそ均等にします。



本書の配布コードにこのタイル並列版を**実装済みとはしていない**。上級プロジェクトの研究課題として、入力タイルの独立性・境界値・出力マージをまず仕様化し、実装後にテスト設計の章のオラクルと完全一致することを確認します。

ハンズオン／確認問題

並列化対象を、(A) 同じ入力上で互いに別の初期配置をシミュレートする、(B) 一つの盤面を分割する、の 2 方式で考え、どちらがアフィン配列の入力共有問題を生むか説明してみましょう。

Chapter 27

GPU 呼び出しと実行レーン

GPU（graphics processing unit）は多数の計算を同時に行える装置です。CPU 向けの処理をそのまま GPU に移せば速くなる、とは限りません。

27.1 ! の意味

Bend では `pow2!(12n)` のように、関数名の直後の `!` で GPU 実行を要求します。呼び出し以下の並列分割が GPU へ配布され、利用できない場合は CPU 側で処理されます。いわゆる手書きの GPU カーネルは通常の導入に必須ではありません。ただし型・所有権・データ構造の相性が速度を決める。

```
import Base

# GPU offloading is requested at the root, not with an explicit kernel.
def pow2(+depth: Nat) -> U32:
  match depth:
    case 0n:
      1
    case 1n+p:
      left right = pow2(p) pow2(p)
      (left + right : U32)

def main() -> IO(Unit):
  IO.print(U32.show(pow2!(12n)))
```

Listing 27.1: GPU 実行を要求するサンプル（10_gpu_tree.bend）

```
bend guide shaders
bend src/10_gpu_tree.bend -o build/gpu_tree
./build/gpu_tree
./build/gpu_tree --gpu off
```

Listing 27.2: GPU 起動、CPU 代替との比較

GPU ビルドには macOS の Metal、Linux の CUDA 12 等が必要です。`gpu_tree.gpu` のような生成物を実行ファイルの側に保持します。`--gpu off` でも結果が 4096 になることを確認します。

27.2 単なる GPU 化で速くなるとは限らない

GPU は同じ形の計算を大量に独立して実行するときに有利だが、メモリアクセス・分岐・転送・同期・タスク数に制約があります。8×8 は非常に小さく、GPU を利用する固定費の方が高い可能性があります。性能向上は**実測**で判断します。

27.3 ライフゲームの GPU 化のための要件定義

1. CPU 版の B_{t+1} と GPU 版の B_{t+1} が、全セルで一致します。
2. 各セルは同じ旧世代の値を読みます。隣接処理が他セルの書き込み順に影響されません。
3. GPU 実行の境界データや配列所有権が Bend の型規則に従う。
4. CPU 側の IO・表示と純粋な更新核を分けておく。
5. 初期のサイズは 8×8、その後 $2^k \times 2^k$ へ段階的に拡大します。

注意

10_gpu_tree.bendは**並列再帰の GPU 例**であって**ライフゲームの GPU 実装ではない**。GPU で動く Life 本体を配布したかのように誤解しないこと。検証には実機・対応コンパイラ・ドライバが必要です。

ハンズオン／確認問題

$64, 4096, 65536$ セルの 3 サイズにおいて GPU 化が有利になりそうな理由と不利な理由を列挙してみましょう。自分の端末で`--gpu off` を使い、出力と起動可否を記録してみましょう。

Chapter 28

性能評価：速さを数字で扱う

速度は「速い気がする」では比較できません。実行条件をそろえて時間を測る**ベンチマーク（benchmark）**の基本を学びます。

28.1　誤解されやすい三つの時間

コンパイル時間、1 世代を更新する実行時間、表示を含む総実行時間は違う。表示が処理の大半を占めると更新核を何倍速くしても全体があまり速くなりません。まず測定の目的を定める。

28.2　比較表を作る

方法	実行時間	正しさ	備考
CPU1 スレッド	自分で記入	Python と比較	入力・測定環境を固定
CPU 並列	自分で記入	CPU1 と比較	並列タイル版が完成してから
GPU	自分で記入	CPU1 と比較	GPU 版が完成してから

28.3　スピードアップと効率

基準時間 T_1、p スレッド時の時間 T_p なら、スピードアップ $S_p = T_1 / T_p$、並列効率 $E_p = S_p / p$。T_p には入力生成・転送・更新・出力のどこまで含めるかを明示します。小さな規模では実行ごとの差が大きいことがあるので、最低 5 回程度を測って中央値を採用します。

28.4　実験ノートの標準書式

1. Bend のバージョン／コミット、OS、CPU コア数、GPU、コンパイラを記録。
2. W, H, g、初期パターン、境界規則、出力の有無を記録。
3. 測定するバイナリを固定してウォームアップの有無を明記。
4. 1・2・4・8 スレッド、GPU ON/OFF の可否と結果を区別します。
5. 全ケースで最終盤面を照合してから速度比較します。

ハンズオン／確認問題

本書の 8×8 と 4096×4096 を比べ、$8WH$ 回の近傍読み出し、WH 回の更新書き込みの規模を求めてみましょう。実測しない数字は**予測**と明記してレポートにまとめてみましょう。

Chapter 29

型を使って条件を表す

依存型（dependent type）は、値に応じて変化する型です。「ただの配列」よりも「長さが決まった配列」のように、仕様を型へ表す発想を紹介します。

29.1　値によって変わる型を理解する

`U32 -> U32` は値を一個受け取り一個返すことだけを述べる。一方「長さ N の盤面に対する安全な添字」「幅 W に適合した行」など、**値に依存する条件**を型の一部に表せるのが依存型の利点です。

Bend の `Nat` は大小関係や構造を使う再帰、`law` における量化、等式型に関わる。`for x: Nat` を使う命題は、単なる試験的な入力群ではなく、すべての自然数を対象にします。

29.2　セルの値を型で制限する設計

現在の `Array<U32>` では、セルが必ず 0 か 1 という要求を型が直接証明しません。検討案は次の二つ。

1. **閉じたデータ型** `Dead/Alive` を使い、どちらかだけを構築可能にします。
2. **値と証拠を組にする**。$v : U32$ に $v = 0 \lor v = 1$ という証拠を付ける。

前者は簡潔だが配列アクセスや GPU 側の表現を新しく検討する必要があり、後者は証明が重くなります。**何を表現時に保証し、何を運用とテストに委ねるか**を整理することが設計の要点。

ハンズオン／確認問題

次の条件を「型に直接書く」「関数の law で述べる」「テストする」のどれに置くか話し合ってみましょう。(1) セルは 0 か 1。(2) 近傍集合に自分を含まない。(3) 4 世代後にグライダーが移動する。(4) GPU が CPU より速い。

Chapter 30

規則の正しさを証明する

命題（proposition）は真偽を論じる文で、**証明（proof）**はそれが正しい理由を示すものです。Bend ではプログラムとして命題と証明を扱います。

30.1　証明したいことを型として書く

`law` で性質を表し、その性質を返す `def` で証明を与える。等号は `{a == b : T}` という**等式の型**であり、通常の `U32.is_eq` という真偽値計算とは別物です。左右が定義を展開して同じ項になるなら `{==}` で示せる。

```
import Base

def choose(is_three: Bool, is_two: Bool, alive: U32) -> U32:
  match is_three:
    case True{}:
      1
    case False{}:
      match is_two:
        case True{}:
          alive
        case False{}:
          0

def rule(alive: U32, +n: U32) -> U32:
  choose(U32.is_eq(n, 3), U32.is_eq(n, 2), alive)

law three_is_alive:
  for cell: U32
  {rule(cell, 3) == 1 : U32}

def three_is_alive(cell):
  {==}

law two_preserves:
  for cell: U32
  {rule(cell, 2) == cell : U32}

def two_preserves(cell):
  {==}

law zero_is_dead:
  for cell: U32
  {rule(cell, 0) == 0 : U32}

```

```
def zero_is_dead(cell):
  {==}

# A proof file can omit main. Run: bend 11_rule_proofs.bend --verdict
```

Listing 30.1: 局所遷移規則の 3 つの law（11_rule_proofs.bend）

30.2 証明している命題を日本語で読む

1. three_is_alive: 任意の U32 のセル値に対し、近傍数が 3 なら結果は 1。
2. two_preserves: 任意の U32 のセル値に対し、近傍数が 2 なら結果はもとの値。
3. zero_is_dead: 任意の U32 のセル値に対し、近傍数が 0 なら結果は 0。

これらは**局所規則の三つの性質**であり、盤面の近傍数計算が正しいことや、同期的に更新されたことを含意しません。また two_preserves は入力を任意の U32 としているため、もし入力値が 5 なら出力も 5 となるが、それを「5 は生存を表す」と考えてはいけません。

```
bend src/11_rule_proofs.bend
bend src/11_rule_proofs.bend --verdict
```

Listing 30.2: 証明モード

--verdict は証明を追加の小さな検査器で確認する機能を使います。実機で導入版の出力を確認します。@unsafe や外国語の効果に頼るコードを証明の一部へ取り込まない。

30.3 意図的に破ってみる

choose の True{} 側を 0 に変えると three_is_alive が壊れるはずです。実験では元のファイルを複製し、失敗が確認できたら戻す。?TODO を証明の代わりにしたものが完成した証明ではないことにも注意。

ハンズオン／確認問題

(1) 3 つの law の量化が何を意味するか日本語に訳してみましょう。(2) いずれかの規則を壊して証明が失敗するか調べてみましょう。(3) one_dies（近傍数 1 なら必ず 0）という第四の law を追加し、{==} で示せるか試してみましょう。

Chapter 31

局所証明から盤面全体の正しさへ

一つのセルの規則が正しくても、盤面全体が正しく更新されるとは限りません。局所的な性質を組み合わせて全体の性質を考えます。

31.1　未証明の箇所を明示する

局所の rule が正しいと証明されても、`neighbors` が 8 方向を正確に読んだか、`fill` がすべての座標をちょうど一度書いたか、`step` が旧盤面だけを参照したかは別の命題です。本書では盤面全体の形式的証明を完成したとは記述しません。

31.2　全体仕様への分解案

1. 座標変換 $i = yW + x$ が範囲内で一意です。
2. 各セル p に対して定義した $N(p)$ がトーラスのちょうど 8 近傍です。
3. $\mathrm{count}(B, p) = \sum_{q \in N(p)} B(q)$。
4. $\mathrm{step}(B)(p) = F(B(p), \mathrm{count}(B, p))$。
5. 上の性質を `evolve` へ帰納して、g 世代の意味と一致させる。

仕様を細かい lemma に分割することと**実装に対応した表現を選ぶ**ことが証明の大部分を占める。抽象的な無限盤面と有限の 8×8 トーラスを混同しません。

31.3　law 駆動の開発

実際の大きな Bend プロジェクトではLAWS.bendとPROOF.bendを分ける慣行があり、実装だけを変更して法則を破ることをビルドで防げる。まず小さい11_rule_proofs.bendで証明の構文を学び、規則・証明を別ファイルへ分割する演習へ進みます。`bend PROOF.bend --verdict` を成功させることが厳密なゲートになります。

ハンズオン／確認問題

全体仕様の 5 つを「証明済み／テスト済み／未着手」に分類し、根拠を一行ずつ書いてみましょう。**型検査が通った**だけで「正しさが証明された」と区別する練習をしてみましょう。

Chapter 32

上級プロジェクト：可変盤面と GPU 化を設計する

最後の発展制作として、盤面サイズの変更と GPU 実装の設計に進みます。ここは完成済みの機能ではなく、自分で設計・検証する課題です。

32.1　どの順番で拡張するか

8×8 固定の教材から先の順番は、**(1) 設定可能なサイズ、(2) 初期パターンの読み込み、(3) 境界仕様の切り替え、(4) リファレンスとの照合、(5) 並列タイル化、(6) GPU 化、(7) 証明の拡張**が妥当です。

32.2　可変サイズにするときの具体的な問題

本書は 64 セルと幅 8 がコードに直書きされます。幅 W と高さ H を導入したら、$W, H > 0$、配列容量との整合、$W \times H$ のオーバーフロー、`Array` の 2 の冪の領域サイズ制約、物理配列の余白の扱い、端のラップをそれぞれ定義します。16×16 で成功したからといって任意サイズが実装できたとは言えない。

32.3　ハロー付きタイル化の擬似コード

```
function prepare_tiles(old_board):
    # Copy read-only 1-cell halos around each tile.
    # Each worker owns its tile + halo snapshot.
    return independent_inputs

function step_parallel(independent_inputs):
    # Process each tile's interior from its OLD halo.
    # Fork/join balanced tile computations.
    return tile_outputs

function merge(tile_outputs):
    # Each output coordinate must be written exactly once.
    return next_board
```

Listing 32.1: 設計用擬似コード。Bend としてそのまま実行不可

独立入力の作成、仕事量の均衡、出力の重複禁止、仕様通りの端処理が成否を左右します。GPU へ持ち込む場合は `bend guide shaders` のデータ表現に合わせて計画を修正します。

32.4　卒業制作の到達基準

1. 8×8 版が Bend で実際に動き、Python の 5 枚と一致します。
2. 新しい初期パターンを 2 種類以上実装し、期待する周期・生存数を検証します。
3. 16×16 以上へ拡張し、トーラス境界のテストを追加します。
4. CPU 逐次版と新しい実装の出力を**盤面全体**で比較します。
5. 実行時間を同一環境で測り、速さだけでなく正しさ・所有権・停止性の記録を残す。
6. GPU あるいは並列版が未完成なら、その理由と具体的なブロッカーを正直に報告します。

ハンズオン／確認問題

上の 6 項目を受け入れテスト表に落とし、`inputs / expected / procedure / result` の 4 列を埋めてみましょう。計算資源が乏しい場合はまず正確な 16×16 CPU 版の完成を目指しましょう。

Chapter 33

量付き Kind と汎用的な型を整理する

ここでは Bend の**種類（kind）**を整理します。型が「値の種類」なら、kind は「型そのものの分類」です。最初にこの二段階を区別しましょう。

33.1 Type と Data の一般形

たとえば 3 は値で、その型は U32 です。さらに、型 U32 は複製できる値を表す Data に属します。このように「値 → 型 → 種類」という段階で見ます。一方、配列を表す型 Array<U32> は Type に属します。これは「配列の要素が壊れやすい」という意味ではなく、配列という値を自由に複製できないことに関係します。



Bend の Type は Kind(&1)、Data は Kind(&2) の略記で、&0 は消去量を表します。型構築子が受け取る値の再利用可能性は Kind(a) と**量**で記述できます。a <&> b は量の小さい方を選ぶ演算で、複数のフィールドを持つ型の所有権の上限を考えるときに使う。

```
import Base

# Kind(&1)=Type, Kind(&2)=Data; List preserves its element kind.
def length(a, -A: Kind(a), xs: List<a, A>) -> Nat:
  match xs:
    case Nil{}:
      0n
    case Con{h, t}:
      (1n + length(a, A, t) : Nat)

def main() -> Nat:
  length(&2, U32, [1, 2, 3])
```

Listing 33.1: 量を明示する List の長さ（23_kinds.bend）

33.2 配列の所有権と複製の条件

Array<U32> は Type に属します。中身の U32 が Data でも**配列そのものの所有権**が自動で Data になるわけではありません。汎用関数では-A: Kind(a) と List<a,A> のように量をパラメータ化でき、実装上の制約を型に持ち込める。

ハンズオン／確認問題

(1) `List<U32>` と `List<U32 -> U32>` の再利用可能性を比較してみましょう。(2) `Kind(&0)`, `Type`, `Data` を「いつ消えるか／何回使えるか」で整理してみましょう。

Chapter 34

証明の基本操作：帰納法・書き換え・反証

数学的な証明を一段進めます。**数学的帰納法（mathematical induction）**は、最初のケースと次のケースへの引き継ぎを示す方法です。

34.1　等式の証明を一歩進める

{==} は正規化して同じ項になった等式を証明します。任意の自然数に関する等式などでは、`match n` による**場合分け**、小さい `p` への再帰による**帰納法**、`%proof : P` による**書き換え**が必要になります。

```
import Base

law plus_zero:
  for x: Nat
  {Nat.add(x, On) == x : Nat}

def plus_zero(x):
  match x:
    case On:
      {==}
    case 1n+p:
      %plus_zero(p) : {1n+Nat.add(p, On) == 1n+_ : Nat}
      {==}

def main() -> {Nat.add(2n, On) == 2n : Nat}:
  plus_zero(2n)
```

Listing 34.1: 自然数についての帰納法（24_proof_induction.bend）

34.2　存在と否定を表す・未完成の証明を扱う

`exs x: T` は**証人と、その性質の証明**を要求します。`{a != b : T}` は、等しいと仮定すれば `Empty` に至る関数の型です。`?name` はゴール表示、`?TODO` は未完成の穴であり、完成の証明を意味しません。`@unsafe` および `def f?` は停止性の保証から外れるので、`--verdict` により実装の型検査と独立に判断します。

34.3 LAWS/PROOF の二ファイル構成

人間が保持する `LAWS.bend` に法則を書き、別の `PROOF.bend` でその `law` を満たす `def` を定義します。どの命題が未証明かを残し、`bend PROOF.bend --verdict` を検査ゲートにします。Life では最初に `rule` の三つの局所性質を証明し、次に近傍集合と同期更新の正しさへ進みます。

ハンズオン／確認問題

(1) `Nat.add(x,0n)=x` の帰納段階を一行ずつ説明してみましょう。(2) `?TODO` を入れると検査結果がどう変わるか確認してみましょう。(3) `law` の命題を変えずに実装を改修する運用案を考えてみましょう。

PART V

解答・補足資料

この Part の目次

35 重要演習の解答と検証方法 75
36 環境・コマンド・参照先 77
37 Bend 2 基本機能監査表 79

Chapter 35

重要演習の解答と検証方法

ここからは解答・付録です。自力で解いてから照合し、単に答えが一致したかだけでなく、理由も説明できるか確認してください。

35.1　18 ケースの局所規則

n	0	1	2	3	4	5	6	7	8
$F(0,n)$	0	0	0	1	0	0	0	0	0
$F(1,n)$	0	0	1	1	0	0	0	0	0

35.2　代表的な添字

$(0,0)\mapsto 0$, $(7,0)\mapsto 7$, $(0,7)\mapsto 56$, $(7,7)\mapsto 63$。$(2,3)\mapsto 26$。$(7,0)$ の右は $(0,0)$ で、$(0,1)$ ではありません。

35.3　グライダーの変位

世代 0 の集合は $\{(1,0),(2,1),(0,2),(1,2),(2,2)\}$。4 回の更新後は $\{(2,1),(3,2),(1,3),(2,3),(3,3)\}$。`tests/test_reference.py` にはこの性質を全セル集合として比較するテストがあります。

35.4　近傍読み出し回数

1 世代で $N=WH$ 個のセルを更新するなら、現行の各セルは `neighbors` の 8 回と現在セルの 1 回で、合計 $9N$ 回の `Array.get` を使います。`Array.set` は N 回。計算量の章の「8 方向参照」と現在セルの読取りを区別すること。

35.5 典型的なバグ

症状	疑う箇所
盤面右端付近だけ違う	x 方向のラップと一次元添字の混同。
ブリンカーが 1 周期で崩れる	旧世代を更新しながら後続の近傍を読んでいる。
すべてのセルが 1 になる	自分自身を近傍として含めた、あるいは条件式の 3 と 2 を間違えた。
`used more than once`	配列を戻り値として引き継いでいない、または `+` の不足。
`expected Data observed Type`	`+Array` とした可能性。配列を共有せず設計を直す。
性能測定が極端に遅い	正規化経路や端末描画を測っていないか確認。

Chapter 36

環境・コマンド・参照先

困ったときに参照するコマンドをまとめています。**再現性（reproducibility）**は、同じ条件で結果を再確認できる性質です。

36.1　コマンド早見表

コマンド	目的
`bend version`	バージョン表示。
`bend guide`	インストール版に一致したガイド。
`bend base`	標準ライブラリの参照。
`bend base Array`	配列周辺の API を個別に調査。
`bend guide shaders`	GPU・並列描画の最適化ガイド。
`bend file.bend --check-only`	型・停止性などのチェックだけ実施。
`bend file.bend`	チェックと実行／正規化。
`bend file.bend -o program`	ネイティブバイナリの生成。
`./program --threads 4`	CPU スレッド数を指定。
`./program --gpu off`	GPU マーク付き処理を CPU で実行。
`bend file.bend --verdict`	追加の証明検証を要求。
`python3 -m unittest discover -s tests`	Python 参照系の自己テスト。

36.2　主要資料

- https://github.com/bendlang/bend
  公式リポジトリ、README、guide、Base、リリース。
- https://github.com/bendlang/bend/releases（公式の最新版・リリース日時）。
- https://bend2.dev/（非公式だが例題の豊富な学習リファレンス。公式 `bend guide` が優先）。
- https://nohzafk.github.io/bend2-from-zero/（Bend2 の失敗例とライフゲームを含む独立教材。実装上の参考情報）。

36.3　検証ログの書式

```
Date: YYYY-MM-DD
OS / Arch:
Bend version:
Bend native clang:
GPU backend:
Commands:
  bend src/08_life_terminal.bend --check-only
  python3 scripts/check_bend.py
Result: PASS / FAIL / SKIP
Error output (if any):
Changes made after checking:
```

Listing 36.1: VERSION_LOG.md の記入例（数値は実測値に置換）

36.4 最後の総合チェック

1. **動作**: 各章のプログラムの出力を自分で確認した。
2. **数学**: B3/S23、トーラス、同期更新を自分の言葉で説明できます。
3. **言語**: `Data/Type`、`+`、`Nat`、`Array.get`、`match` の制約を説明できます。
4. **並列**: fork/join と GPU 実行の違い、計算負荷の均衡を説明できます。
5. **証明**: law の意味と、その法則が**保証しないこと**を説明できます。
6. **再現性**: ソース・バージョン・コマンド・テスト結果を再現可能な形で残した。

要点

本書の境界線：逐次ライフゲームは Bend 2.0.37 で 26 ファイルの型検査と、5 世代全セルのオラクル比較に合格しました。並列ツリーと GPU ツリーは独立の練習例です。ライフゲームの CPU 多コア版・GPU 版・GUI 版・盤面全体の形式証明は今後の拡張課題です。

Chapter 37

Bend 2 基本機能監査表

最後に、Bend 2 の基本機能の学習範囲を確認します。本文で解説済みの項目と、実装・実行確認がまだの項目を混同しないようにしましょう。

37.1　監査基準と凡例

2026 年 10 月 16 日に公式 `guide/GUIDE.md`、公式 `README.md`、公開 `base.bend` を突き合わせた。下表の**本文・コード**は文章解説と独立`.bend`例が存在すること、**本文・課題**は構文・用途・実習課題を与えたもの、**発展**は高度な実装を完成したと主張していないものを指す。Bend コンパイラによる実行確認状況とは別です。

公式ガイドの基本項目	対応	掲載内容と限界
型・関数・注釈・演算子	本文・コード	Part I：U32/Nat、関数、リテラルと型注釈。
ADT・コンストラクタ・match	本文・コード	Part I：Cell、パターンマッチ。
再帰・停止性・unsafe	本文・課題	List、燃料、停止性、unsafe の保証限界。
クロージャ・高階関数	本文・コード	adder とアフィンな関数値。
テンプレート（~）	本文・コード	twice と閉じた引数・インスタンス化。
数量（-, 無印, +）	本文・コード	所有権と再利用の制約、違反例。
Kind, Type, Data, Quant	本文・コード	Kind(a)、量の演算、List の種類。
List, Array, Pair	本文・コード	構造再帰、配列更新、ペア返却。
Bool/Cmp, 数値、Char/String	本文・課題	主要型、演算子、変換、bit 操作。
Maybe, Result, Monad/do	本文・コード	Maybe の do 例、Result の意味と導入課題。
Map, Set	Map にコード・Set は課題	Map の所有権付き API と Set の標準ライブラリ調査。
モジュール／ import	本文・コード	別ファイルへの分割と alias。
IO、args、例外的失敗	本文・課題	IO.print、IO.args、IO.try、env 等。
IO 並行性（fork/join/Chan）	本文・コード	fork/join 例、Channels の概念と制約。
CPU 並列呼び出し	本文・コード	独立の並列木例。Life 本体は未並列化。

公式ガイドの基本項目	対応	掲載内容と限界
GPU 呼び出し（!）	本文・コード	独立の GPU 木例。Life 本体は未 GPU 化。
App、Image、Event	本文・コード	ウィンドウ入門例。Life の GUI は未完成。
ファイル、ネット、プロセス、音声	File にコード・他は課題	ファイル作成とハンドル管理。残りは API 調査。
C/JS ホスト効果、JS モジュール	本文・課題	仕組み、ビルドと実験。FFI 実装は応用。
law、等式、帰納法、verdict	本文・コード	局所規則、帰納法、証明の到達範囲。
BendHub publish、各種出力	本文・課題	公開手順の意味、ネイティブ/C/JS/mjs。

37.2 網羅性の限界：基本構文と全 API は別

この表が意図する**基本機能の網羅**とは、公式ガイドが独立の見出しとして扱う概念を学習できる状態にすることです。Base の約数千行に及ぶ全関数、ネットワークの細かな挙動、GPU 最適化、独自 FFI、BendHub ログイン・公開の実演まで全て本文で完了したとはしていない。必要になったときは `bend base`、`bend guide effects`、`bend guide shaders` と現行ソースを参照します。

注意

検証範囲の注意：CI 上で全 26 ファイルの型検査と、逐次ライフゲームの出力比較には合格しました。ただし、個別サンプルの GUI・GPU 実行、全ての標準ライブラリ API、`--verdict` による証明検査は網羅していません。「本文・コード」はあくまで掲載範囲を示します。

37.3 追試チェックリスト（Bend を導入したら）

1. `bend version` を記録し、全 `src/*.bend` のうち `INVALID` を除いて `bend ... --check-only` を実行します。
2. `src/08_life_terminal.bend` で差分テストを行います。Python と全世代全セル一致を確認します。
3. IO、テンプレート、モジュール、Kind、App は単独サンプルを実行し、実機とコンパイラバージョンを記録します。
4. `--verdict` で証明例を確認し、**安全な証明**と**部分的な検査**を区別します。
5. ターゲットごとの差（ネイティブ、GPU、JavaScript）と外部の必須ライブラリを記録します。